

K4510

USER'S AND PROGRAMMER'S GUIDE

a fantasy 8/16-bit computer

Michael Longval
project orchestrator

Draft version: alpha-0.5+219
CPU 45GS10 · VICKY · MELODY · 256 MB RAM

Date: 14.09.2026

This is alpha documentation. It describes a machine that is still being built, and it is written alongside it. Everything in it — commands, keys, registers, the layout of files, the names of things — is subject to change, and some of it will have changed between this printing and the machine you are running. Where the two disagree, the machine is right and this book is behind it; Appendix B says how to tell us. The version on the cover is the build this book was made from.

Contents

I	User's Guide	1
1	The Machine	2
1.1	What is the K4510?	2
1.2	One machine, three ways to run it	3
1.3	Building the machine	4
1.4	First boot	5
1.5	The keyboard	6
1.6	The F7 menu	7
1.7	The menu file: what the menu shows	9
1.8	Placement, the side panel, and F8	14
2	The Shell	16
2.1	This book, on the machine	16
2.2	The disk	17
2.3	Running things	19
2.4	The file managers	20
2.5	SWAP: running one thing from inside another	21
2.6	At the prompt	21
2.7	Aliases	22
2.8	The status bands	22
2.9	The network	22
2.10	CAPSLOCK	24
2.11	Scripts and the boot file	24
2.12	The monitor	25
2.13	Colours and palettes	25
2.14	Housekeeping	27
2.15	When STARTUP.BAT is the problem	28
2.16	When something goes wrong	28

3	Every Command	29
4	EhBASIC	36
4.1	Ten minutes of it	36
4.2	The machine's own statements	36
4.3	Hardware sprites	37
4.4	The * escape	39
4.5	Editing the program in VI	39
5	Microsoft BASIC, 1977	40
5.1	Saving and loading	40
5.2	The * escape	41
5.3	What it does not have	42
6	The Tube: BBC BASIC	43
6.1	Graphics and sound	43
6.2	Sprites	44
6.3	The terminal: JIM	45
6.4	Star commands	46
7	Forth	48
7.1	The machine at your fingertips	48
7.2	The assembler	48
7.3	Manners	49
8	LOGO	50
8.1	The words	52
8.2	Procedures	52
8.3	Files	53
9	CP/M: the Z80 Second Processor	54
9.1	Drives are folders	54
9.2	Starting a program from the shell	55
9.3	Typing a CP/M program's name directly	56
9.4	Software	56

10 Pascal	58
10.1 Turbo Pascal 3, on CP/M	58
10.2 Mad Pascal, the cross-compiler	59
11 The Editors	64
11.1 EDIT	64
11.2 VI	65
11.3 Editing from inside a BASIC	68
12 RX: the Machine's REXX	69
12.1 Why REXX, and what ARexx was	69
12.2 The three environments	70
12.3 Ports: the file is the mailbox	70
12.4 What is in /LANG/RX	70
13 The Linux Underneath	72
13.1 ! — the Linux prompt, from the machine's prompt . . .	72
13.2 PAS and CC: compiling on the machine	73
13.3 What is on the other side	74
13.4 On a stick	75
13.5 Beside another system	75
13.6 A Tektronix beside the machine	76
13.7 From another computer	77
II Programmer's Guide	78
14 Memory	79
14.1 The 64 KB view and the 256 MB behind it	79
14.2 The CPU view, unmapped	79
14.3 Programs	80
14.4 System calls	80
14.5 RAM under the ROM	81
14.6 Sideways ROM	81
14.7 RAM under the I/O page	82
14.8 Programs bigger than the window	82

15 The I/O Page	83
15.1 A split screen: SHEILA	83
A The Registers	86
A.1 The I/O page	86
A.2 VICKY, SHEILA and the sprites	91
A.3 The network	94
A.4 JIM, the terminal	96
A.5 Memory and the far view	97
B Filing an Issue	98
B.1 Before you try again	98
B.2 BUG writes the report	98
B.3 Where it goes	99
C The Sound, and What It Took	101
C.1 Four chips, because it is a fantasy	101
C.2 A SID is not a CPU	102
C.3 “Choppy choppy choppy” — which was not the sound	102
C.4 Silence is not free	103
C.5 The ring that started empty	103
C.6 Chips that play on without the machine	103
C.7 The meter that read zero	104
C.8 A machine keeping somebody else’s time	105
C.9 Where it stood, when it was decided	106
C.10 What was decided	106
C.11 And the moral, once more	107
Disclaimer	109
Thank You	111
The heart of the machine	111
The tongues	111
Letters on the screen	112
The bare metal, which the machine no longer stands on	113
The workshop	114
Heritage	114

And whoever is missing	116
Licences	117
The project itself	117
Code inside the machine	117
Fonts	118
The Linux the machine boots on	119
Build tools	120
What is deliberately not shipped	120
If you redistribute the machine	121

Part I

User's Guide

Chapter 1

The Machine

The K4510 is a fantasy computer: a machine that never existed, built the way 1985 might have built it with no budget committee in the room. It is not an emulation of any real computer. Its CPU, video chip, sound, and operating software are its own; the parts it borrows — a 6502-family instruction set, the AdLib's FM chip — it borrows openly and then outgrows.

1.1 What is the K4510?

- **CPU: the 45GS10** — the MEGA65's 45GS02 instruction set (a 4510 with the Q pseudo-register and 32-bit flat addressing) plus this machine's own memory management: bank registers, a far-call gate, and RAM under the ROM. Its clock is whatever the host it runs on can hold at sixty frames a second with no gaps in the sound: the machine is a fantasy and its timings are suggestions. The clock is a setting (F7, under Machine — ten steps from 202.5 MHz down to 10). A host nobody has measured runs at 40.5, the MEGA65's number; SETUP measures it properly, with the sound and the picture really running, and the machine keeps the answer for that host so later boots pay nothing. It never second-guesses a clock you chose yourself. A desktop of the last ten years holds a hundred megahertz or more. INFO reports the clock in force, read live, and BENCH is there when you want the whole picture.
- **Memory: 256 MB**, flat, 28-bit. The CPU sees 64 KB at a time; everything else is one instruction away.
- **Video: VICKY** — 640×480, 256 colours from a 24-bit palette, four layers, 128 sprites, a blitter that draws lines and filled triangles, and a display-list coprocessor named SHEILA. Smaller modes (640×240,

320×240, 320×200, 160×200) are drawn into the same glass, doubled and centred, so the raster is always 480 lines however few of them the picture uses.

- **Sound: MELODY** — an OPL2, the Yamaha YM3812, at \$D480: nine FM voices wired the AdLib's way, an address port, a data port and a status register you poll. Any AdLib register list or instrument patch therefore means what it says on this machine. A four-channel sound sequencer at \$D5E0 plays through it, in the BBC Micro's idiom, which is what BBC BASIC's SOUND and Mad Pascal's Sound use. A floating-point MATH unit sits beside it, which the BASICS and LOGO lean on. (The machine had four SID chips until September 2026. Appendix C is the whole story.)
- **K/OS:** a shell with directories, in ROM — and on the disk, one word away each, seven languages: EhBASIC with graphics and sprites, Microsoft's own BASIC of 1977, BBC BASIC on the Tube, Forth, LOGO, CP/M on a Z80, and RX, the machine's REXX. Two compilers are one word away as well: PAS and CC turn a Pascal or C source in the directory you are standing in into a program beside it (Chapter 13).

1.2 One machine, three ways to run it

The computer is the **K4510**: the 45GS10, VICKY, SHEILA, MELODY and K/OS. There is one of it. What changes is what it is standing on.

As a whole computer, from a stick. Write the image (`linux/build-live.sh`) to a USB stick, boot a spare laptop from it, and the machine is the machine: a minimal Debian that exists only to hold it up, loaded into RAM, keeping its settings and your files on the stick and never touching the laptop's own drive.

As a whole computer, installed beside another system. `linux/install-k4510.sh` copies the same system from the stick onto a partition of a computer's internal disk and adds it to that computer's boot menu, so the machine is one choice of two when it starts. The other system is not changed, and the computer boots whichever of the two was used last (Chapter 13).

Either way, the Linux underneath is not hidden. The cross-compilers, git, an editor and a second terminal are on it, and the machine can reach them (Chapter 13); but it is furniture. Switch on and you are at the [/HOME] prompt.

As a window. Run the same program on a Linux desktop and you get the same machine in a window, which is how it is developed and tested, and how most of the figures in this book were taken. There is also a sandboxed container flavour (`linux/podman.sh`) that is shown nothing of your computer but the display, the sound, the game controllers and one shared folder.

Same ROM bytes, same software, every way. Where the desktop window and the whole computer differ, the book says “on a desktop” or “on the K4510’s own Linux”. (Until 2026-09-07 there was also a bare-metal edition for the Raspberry Pi 3B+; it was retired so that every feature is built once. A Pi still runs the machine under Linux like any other computer; the tag `alpha-0.5` is the last tree with the bare-metal edition in it.)

1.3 Building the machine

On a **Linux desktop**, three lines build the whole machine:

```
git clone https://github.com/mlongval/k4510
cd k4510 && ./setup.sh
./k4510
```

`setup.sh` installs the compilers and SDL2, builds everything, and runs the machine’s test suites.

Or: the whole computer

```
sudo ./linux/build-live.sh
```

makes the bootable image: a small Debian, the machine and its compilers, about half an hour the first time. Write it to a USB stick with `dd` and boot from it; to put it on a computer’s own disk instead, boot that

computer's usual Linux with the stick in and run `sudo ./install-k4510.sh` from it. Chapter 13 has both.

What it needs

The machine asks little of a computer, and most of what it asks is for the Linux under it:

Processor	64-bit x86, two cores; a Core 2 is enough to run it, and what the processor decides is only the clock SETUP settles on
Memory	2 GB; 4 is comfortable. The whole system is loaded into RAM at boot (about 0.9 GB), and a running machine uses 1.7 GB in all, the emulator itself 90 MB of it
Graphics	any Intel or AMD chip Linux drives natively (KMS); no desktop is used
Screen	640×480 or more
Firmware	UEFI or BIOS, either
Storage	a 2 GB USB stick, or a 2 GB partition

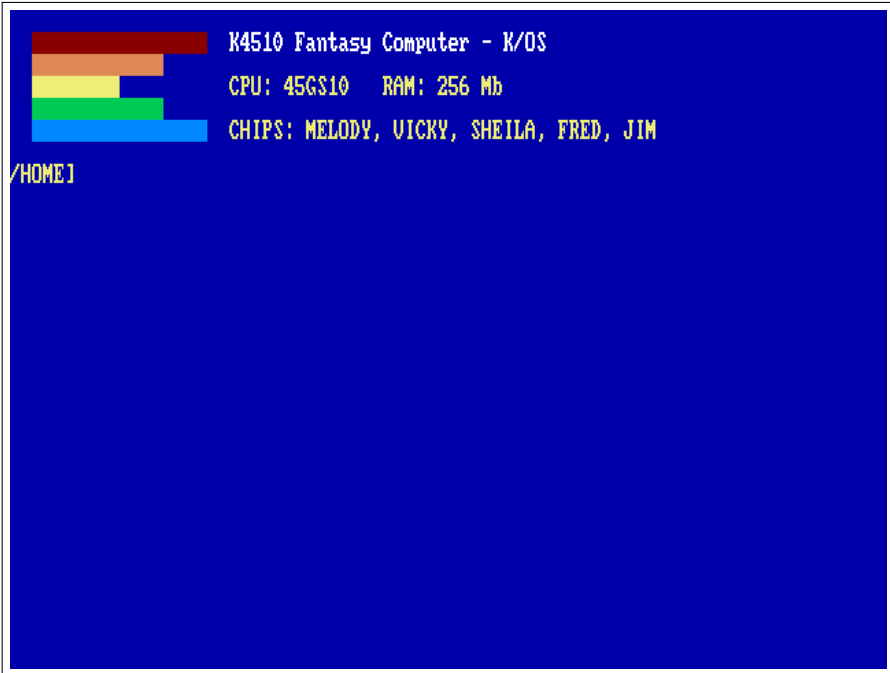
In a window on a desktop it needs only what the desktop has already: SDL2 and a few hundred megabytes of memory.

1.4 First boot

Switch it on (on a desktop: `./k4510`; otherwise, power up). The machine's logo is on the screen while Linux starts, and then the machine greets you in yellow on blue:

The `/HOME]` at the bottom is the shell prompt — the part before the `]` is the directory you are in, and the machine starts you in `/HOME`, which is yours. Type `HELP` for the commands, `INFO` for the machine's full self-description, and `DIR` to see your files. The first time on a new computer, type `SETUP` and let the machine measure it.

Keys: Escape is RUN/STOP (stops a BASIC program), Ctrl-C is STOP, Shift+Escape quits the emulator. Reset is a chord, the way it was Commodore+Restore on the C64: Super+PageUp (the Windows or



The boot screen: five stepped colour bars, then the machine, its CPU, its memory and its chips — and no clock, because the machine no longer has one speed. Everything else, the clock in force included, is one INFO away, and BANNER prints this again.

Command key and PageUp) — one key cannot do it by accident. F7 opens the settings menu; F8 pauses the machine and turns the keyboard into a debugger's. PrtSc takes a screenshot — the screen flashes, and the picture is in shots/ beside the emulator.

1.5 The keyboard

The keyboard is the one in front of you, and F7 → Input → *Keyboard layout* says what its keys mean: US, US-International, Canadian French, French, German, Spanish, UK or Italian — or *Host*, the first choice, which leaves the question to the desktop or Linux the machine is running on. A

layout chosen here takes effect at once, with its AltGr and its dead keys: on US-intl, ' then e is é, the dead key twice or before a space gives the mark itself, and a ' or " before a letter that takes no accent types itself. On the K4510's own Linux the Linux consoles follow the same choice. A letter the machine's character set (the IBM PC's) cannot draw is not offered. For programming, plain US has no dead keys to step round.

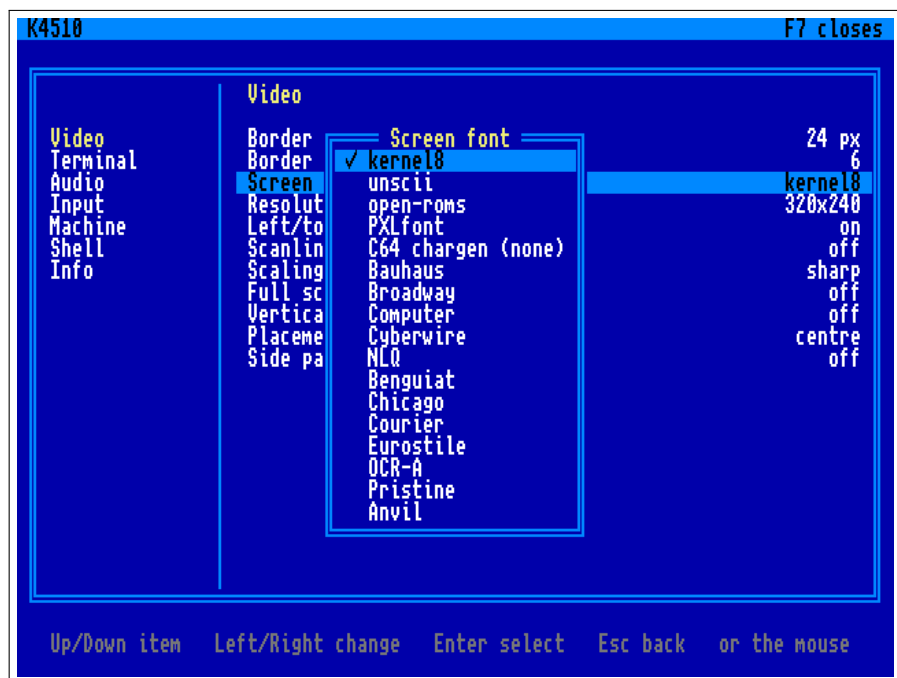
Caps Lock is Ctrl, beside it, makes the Caps Lock key a second Ctrl, where the old Unix keyboards had it; the shell's own CAPSLOCK command (Chapter 2) is how you get capitals without it.

1.6 The F7 menu

F7 (unshifted; Shift+F7 still reaches BBC BASIC) freezes the machine and takes the whole screen, in the manner of the C64 Ultimate's: the categories down the left, the settings of the chosen one on the right, and a line at the foot saying what the keys do in whichever pane holds the cursor. Sound stops, and closing the menu resumes exactly where it froze. The menu draws with its own font and never touches the machine's memory, so it opens even when a program has wrecked the screen.

Up and Down move; Enter or Right crosses from the categories to the settings; Left and Right step a value; Escape goes back a pane, and closes at the categories; F7 closes from anywhere. The mouse works too. A setting with a list of choices opens that list over the panes, and *applies as the cursor passes each one* — the screen font swaps under the menu so you can see it — with Escape putting back whatever was there when the list opened.

Video the border; the screen font (a live swap of the character set — *kernel8*, *unscii*, *open-roms* and *PXLfont*, twelve of Damien Guard's ZX Origins faces, and *C64 chargen* if you have put a converted Commodore character ROM in /SYSTEM/ETC/chargen.bin, which /SYSTEM/ETC/README.TXT explains); *resolution* and *left/top margin*, which are the MODE command's two knobs; *scanlines*, a dark line between each of the machine's, drawn exactly rather than overlaid; *scaling*: *sharp*, *soft*, or *sharp-fit* (hard pixels *and* a whole-number multiple, so every pixel is the same size on the glass); full screen;



The F7 menu: categories on the left, the Video page on the right, the screen-font list open over them.

vertical sync, off to begin with — see the aside below; and *placement* and *side panel*, which are the section after next. The figures in this book are taken with the effects off.

Terminal *status bands* and how many rows each takes (Section 2.8), and the clock and date format they print.

Audio volume.

Input the reset chord; which key opens the menu; whether a click captures the mouse pointer, and whether the host's pointer shows over the picture; and the keyboard, above.

Machine *save state* and *load state*, four slots each (the whole machine — CPU, every used page of the 256 MB, VICKY, the devices, JIM — to k4510-slotN.k4s beside the settings file; the Tube co-processor

is not in the file and is stopped by a load); reset; power cycle; stop the Tube; quit; *CPU clock* — the ten steps, live; choosing one switches *Auto clock* off, because a clock chosen by hand is not to be second-guessed. *Auto clock* on uses what SETUP measured on this host. On the K4510's own Linux there is a last row, *Shut down the computer*.

Shell whether an unknown word at the prompt may run a CP/M .COM (Chapter 9) — off to begin with, on purpose: D typed for DIR should not start a Z80 program; and whether /STARTUP.BAT runs at power-on, which is the way out of a bad one (Section 2.15).

Info the version and the exact build, the ROM, the files, the host — the K4510's own Linux, or a window on a desktop — and the battery, where there is one.

Host on the K4510's own Linux only: the computer's name, its network address and Tailscale's; *Lid closed*, which is *keep running* to begin with — the machine goes on with the lid down — or *suspend*; a *Wi-Fi / network setup* row that opens the network settings on a spare console, and a row that telnets into the Linux for you.

Leaving the menu writes the settings to k4510.cfg beside fs/ — a plain key = value file you may edit; comments and keys it does not know are kept, which is how a settings file survives the machine changing under it.

Vertical sync is off, and that is deliberate. With it off the machine keeps its own 60 Hz and presents a frame when it is ready; the cost is tearing. With it on, the pacing belongs to your display, and on a host whose refresh is not exactly 60 that costs frames — and frames are sound here (Appendix C). Neither answer is right for every host, which is why it is a row and not a decision.

1.7 The menu file: what the menu shows

Beside k4510.cfg is a second file, k4510-menu.cfg, and it decides not what the settings *are* but which of them the menu *offers*. The machine writes

it out in full the first time it starts — every category and every row, each marked show — so the file is itself the list of what can be changed:

```
[K4510]
Audio           = show
...
[Video]
Border width    = hide
...
[Locks]
linux           = open
consoles        = open
```

- hide on a row takes it out of the menu, and a category with nothing left in it goes too; hide under [K4510] takes a whole category away. A hidden setting keeps the value it has in `k4510.cfg`: it is not reset, only out of reach.
- `linux = locked` shuts every door into the Linux underneath: !, SSH, and the telnet row of the Host menu. `PAS` and `CC` still compile. `consoles = locked` makes `Ctrl+Alt+F2` to `F6` do nothing.
- Names are the menu's own, in either case; # starts a comment; a row the file does not name is shown. The file is read when the machine starts, so a change takes effect at the next start.

The file lives outside the machine's disk, so nothing running on the machine can undo it. On the K4510's own Linux it is in `/home/k4510/k4510/`; once Linux is locked, edit it over `ssh` from another computer.

Every row

The list below is generated from the menu's own source, with the name each setting has in `k4510.cfg`, its choices and where it starts.

Video

Border width

video.border

0 to 64; to begin with, 0

Border colour	<i>video.border_colour</i>
0 to 15; to begin with, 6	
Screen font	<i>video.font</i>
kernel8, unscii, open-roms, PXLfont, C64 chargen, Bauhaus, Broadway, Computer, Cyberwire, NLQ, Benguiat, Chicago, Courier, Eurostile, OCR-A, Pristine, Anvil; to begin with, kernel8	
Resolution	<i>video.mode</i>
640x480	
Left/top margin	<i>video.margin</i>
on, off; to begin with, off	
Scanlines	<i>video.scanlines</i>
off, light, medium, heavy; to begin with, off	
Scaling	<i>video.smoothing</i>
sharp, soft, sharp-fit; to begin with, sharp	
Full screen	<i>video.fullscreen</i>
on, off; to begin with, off	
Vertical sync	<i>video.vsync</i>
on, off; to begin with, off	
Placement	<i>video.placement</i>
centre, left, right; to begin with, centre	
Side panel	<i>video.panel</i>
off, registers; to begin with, off	

Terminal

Status bands	<i>term.bands</i>
on, off; to begin with, off	
Top band rows	<i>term.band.top</i>
0 to 10; to begin with, 1	
Bottom band rows	<i>term.band.bottom</i>
0 to 10; to begin with, 2	
24-hour clock	<i>term.clock24</i>
on, off; to begin with, on	
Date format	<i>term.datefmt</i>
DD.MM.YYYY, YYYY-MM-DD, MM/DD/YYYY; to begin with, DD.MM.YYYY	

Audio

Volume

audio.volume

0 to 100; to begin with, 80

Input

Reset chord

input.reset_chord

Super+PageUp, Ctrl+PageUp, Alt+PageUp, Ctrl+Alt+Del; to begin with, Super+PageUp

Menu key

input.menu_key

F7, F8, F11, Pause; to begin with, F7

Mouse capture

input.mouse_grab

on, off; to begin with, off

Mouse pointer

input.mouse_pointer

on, off; to begin with, on

Caps Lock is Ctrl

input.caps_ctrl

on, off; to begin with, off

Keyboard layout

input.kbd_layout

Host, US, US-intl, Canada-FR, France, Germany, Spain, UK, Italy; to begin with, Host

Machine

Save state

4 slots, each showing when it was written

Load state

4 slots, each showing when it was written

Reset

does it

Power cycle

does it

Stop the Tube

does it

Quit the emulator

does it

CPU clock*cpu.clock*

202.5 MHz, 162 MHz, 121.5 MHz, 81 MHz, 60 MHz, 40.5 MHz, 30 MHz, 20 MHz, 15 MHz, 10 MHz; to begin with, 40.5 MHz

Auto clock*cpu.auto*

on, off; to begin with, on

Shut down the computer

does it; only on the K4510's own Linux

Shell

CP/M .COM by name*shell.cpm_com*

on, off; to begin with, off

Run STARTUP.BAT*shell.startup*

on, off; to begin with, on

Info

Version

shows

Build

shows

ROM

shows

Files

shows

Host

shows; only on the K4510's own Linux

Battery

shows

Host

Only on the K4510's own Linux.

Name

shows

Address

shows

Tailscale

shows

Lid closed

host.lid

keep running, suspend; to begin with, keep running

Wi-Fi / network setup

does it

Telnet into the host

does it

1.8 Placement, the side panel, and F8

A 4:3 picture on a 16:9 screen leaves a third of the glass empty. *Placement* (centre, left, right) puts the picture against one edge instead of in the middle, and *side panel* fills what is left — at present with one thing, the machine as the emulator sees it. A panel with a centred picture makes no sense, so turning the panel on forces the picture left.

The panel is a strip twenty-six columns wide taking the whole height of the window, one fact to a line: the program counter, A X Y Z, the stack pointer, the B register, the flags, the next few instructions disassembled, VICKY's mode and the raster line it is on, the eight bank registers with the engaged ones lit, the audio gaps, the frame counter and the frame rate. It never reads through the I/O page — a read of \$D100 would pop a key off the keyboard, and an instrument that changes what it measures is not one.

F8 pauses the machine, and then the keyboard is the debugger's:

Key	Does
F8	pause, and again to run on
Space	one instruction
L	the rest of this scanline
F	the rest of this frame
D	write a dump — dumps/dump-NNN.txt, the same file DUMP writes, and arm the instruction recorder so the next one has history
T	start or stop an instruction trace to /SYSTEM/LOG/TRACE.TXT: one line per instruction with the registers after it, stopping itself at 200,000 lines

Every other key is swallowed while the machine is paused. The panel shows the legend and the state: which scanline is next, whether the trace is running and how long it is, and the number of the last dump.

Chapter 2

The Shell

The shell is what the machine boots into. It is a file manager, a program launcher and the front door to everything else, and every command here also works from a BASIC with a * in front. Type HELP for a summary on the screen — the text it prints is itself a file on disk, /SYSTEM/ETC/HELP — and see Chapter 3 for every command with a line about each. This chapter is how the shell behaves.

2.1 This book, on the machine

BOOK is this handbook, on the machine it describes — no browser, no printer:

BOOK	the contents
BOOK 3	chapter 3, every command
BOOK SHELL	the chapter whose title says SHELL

The arrows, PgUp and PgDn (or Space) and Home and End move through a page. Tab chooses the next link and Shift+Tab the one before; Enter follows it and Backspace comes back, as far as eight pages. / finds a word further down, and n finds it again. Q or Escape leaves. A link marked *[picture]* is one of this book's screenshots: Enter shows it full screen, drawn by the machine as the machine drew it, and any key comes back to the page.

The pages are made from the same source as this book and its web edition, at the same time, so the three never disagree. They live in /SYSTEM/DOC, one to a chapter, in Gemini text — a small, open format where a line's first characters say what it is: # a heading, => a link, * an item in a list. So TYPE reads them too, and so does any Gemini reader on another computer.

2.2 The disk

Files live on the host: the `fs/` directory beside the emulator. The machine sees that directory as `/` and nothing above it. The root has six folders and one file in it, and that is fixed — everything else lives one level down:

<code>/SYSTEM</code>	the machine's own, in three parts: BIN the tools (VI, TYPE, MONITOR, RANGER, SETUP, BUG...), ETC what it reads (HELP, VI.SAMPLE, STARTUP.SAMPLE, PALETTES/, a <code>chargen.bin</code> if you add one), LOG what it writes (BENCH-*, TRACE.TXT, BUGREPORTS/)
<code>/LANG/name</code>	one folder per language: its runtime, a README and an EX/ of examples. PASCAL and C keep the sources <i>beside</i> the programs they compile to
<code>/APPS/name</code>	one folder per program, with its data beside it — LODÉ's levels, CHESS's openings, OPLPLAY's tunes
<code>/HOME</code>	yours. The prompt starts here
<code>/CPM</code>	the Z80's drives, A to P, one folder each (Chapter 9)
<code>/MNT</code>	things from outside: SHARE, the folder the container flavour is given, and anything you MOUNT
<code>/STARTUP.BAT</code>	yours; run at power-on (Section 2.15)

The search path, which is why you never type a path

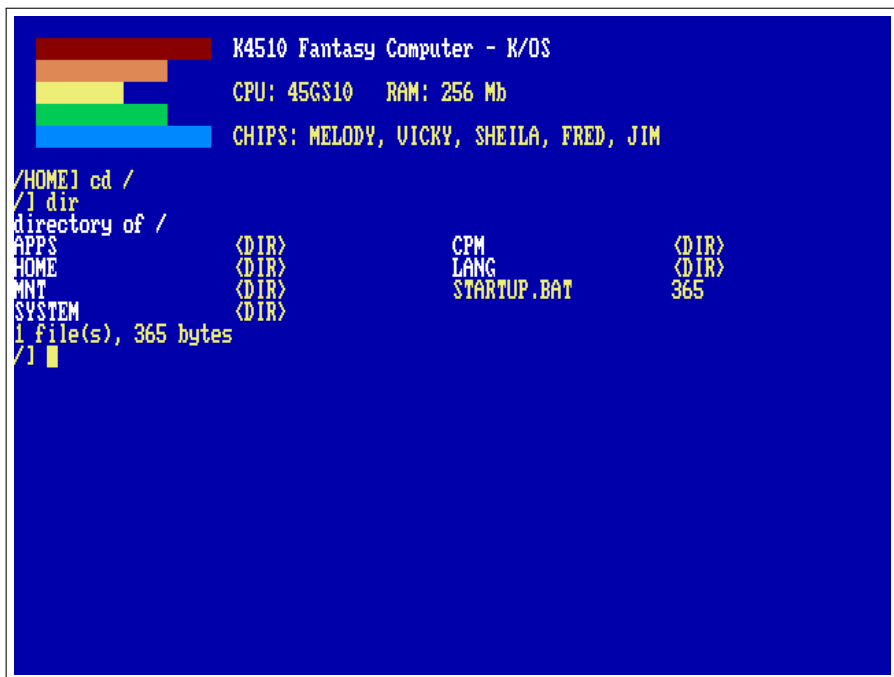
A bare name not found where you are is looked for, in this order, in `/SYSTEM/BIN`, then in `/APPS/NAME/`, `/LANG/NAME/` and `/HOME/PROJECTS/NAME/` — where *NAME* is the name you typed without its extension, upper-cased, because on this disk **the name is the folder** — and then by extension: a `.BAS` in `/LANG/EHBASIC/EX`, a `.BBC` in `/LANG/BBCBASIC/EX`, a `.RX` in `/LANG/RX`, a `.PAS`, `.C` or `.prg` in the two compiled languages' folders. So SKYFIRE, EHBASIC, RANGER and HELLO run from wherever you are standing, and RUN "INVADERS.BAS" inside EhBASIC still finds it. A name is matched regardless of case when the exact one is absent, and `..` never climbs above `/`.

```

DIR -a *.BAS      LS is the same; -a shows the hidden names
CD /LANG/LOGO    the rest of the line is the name, spaces and all
TYPE README.TXT  a screenful at a time; Esc or Q stops
CP README.TXT /HOME

```

Names that begin with a dot are hidden; DIR -a shows them, and DIR -l lists one name to a line. DIR also takes a directory to look in, or a pattern to match: DIR APPS lists that folder without going there, DIR *.PAS and DIR ?.TXT match here (* any run of characters, ? any one).



```

K4510 Fantasy Computer - K/OS
CPU: 45GS10  RAM: 256 Mb
CHIPS: MELODY, VICKY, SHEILA, FRED, JIM

/HOME] cd /
/l dir
directory of /
APPS          <DIR>          CPM          <DIR>
HOME          <DIR>          LANG          <DIR>
MNT           <DIR>          STARTUP.BAT   365
SYSTEM        <DIR>
1 file(s), 365 bytes
/l █

```

DIR at the root: the disk's whole shape in six folders and one file. Directories come first, in white; files carry their sizes; two columns.

Nothing is deleted in a hurry

RM moves a file to /.TRASH rather than destroying it. RM -f is the one that really removes. DELETE -l lists the trash, DELETE -r name puts one back, and DELETE -e empties it; the file managers' own delete uses the same

trash. RENAME and CP refuse to overwrite an existing file, and take `-f` when you mean it — the host's own habit is to overwrite without a word, and that habit has cost this project a file.

2.3 Running things

`RUN balls.prg` runs a program; so does `RUN balls`, and so does plain `BALLS` — an unknown word is tried as a program on disk before the shell gives up, *with its arguments*, the way REXX did it on the mainframes. A program reads its arguments through the `ARGS` system call; `SAY` is the demonstration:

```
SAY HELLO FROM THE DISK
```

prints `HELLO FROM THE DISK` — there is no `SAY` command, only a `say.prg` in `/SYSTEM/BIN`. The same rule takes an `.RX` script (Chapter 12), so a REXX program is a command too. Only a program is run this way: typing the name of a text file at the prompt does not load it over the machine.

Several of the machine's commands are programs in exactly this sense — `TYPE`, `VI`, `SETUP` and the monitor among them — and you cannot tell from the prompt which is which, which is the point. The disk is the host's (and on the K4510's own Linux, in RAM), so a program starts as quickly as a word in the ROM.

Seven words open whole languages: `EHBASIC` (Chapter 4), `MSBASIC` (Chapter 5), `BBC` (Chapter 6), `FORTH` (Chapter 7), `LOGO` (Chapter 8), `CPM` (Chapter 9) and `RX` (Chapter 12).

What is on the disk

`/APPS` holds the programs the machine ships with, one folder each, and any of them runs by name from anywhere:

LODE	a lode-runner: dig, climb, collect the gold. The levels are text files in its folder and E on the title card is a level editor
SKYFIRE	a formation shooter, in the Galaxian rules
FLUFFY	a platformer: variable jump height, stomping, spikes
BOMBER	a bomb-party arena
CHESS	a chess set with an engine; it can also be driven from a script (Chapter 12)
TINY	a dungeon crawl
OPL2	the sound chip, all nine voices
OPLPLAY	plays .OPL streams from its TUNES/
MANDEL	the set, drawn by the machine
CUBE	a rotating solid
BALLS	the sprites, all of them at once
ANSIDEMO	what JIM can draw
SEGDEMO	two overlays sharing one address

2.4 The file managers

Two of them, because they are two different ideas about what a file manager is for, and neither is a compromise.

KOMMANDER is the one with two panels and function keys along the bottom — Tab moves between the panels, and the keys copy, move, make a directory and delete. RANGER is the other tradition: three miller columns, what is under the cursor previewed in the right-hand one, and vi's fingers (hjk_l, gg, G, / to search).

Both share the rules that matter:

- **Enter on a directory** goes into it.
- **Enter on a program** — a .prg, or a CP/M .com — leaves the file manager and runs it, because the output of a program belongs on the machine's screen and not inside a browser's frame. A .com starts CP/M to do it.
- **Enter or F4 on a text file** opens it in VI or EDIT, and you come back to where you were standing.

- **DD deletes** to /.TRASH, the same trash the shell's RM uses, so **DELETE -r** brings it back.

2.5 SWAP: running one thing from inside another

A program is loaded where the last one was, so starting one from inside a BASIC would ordinarily flatten the program you were writing. SWAP command puts the caller away first — the whole of the CPU's memory, and the screen — runs the command on a clean machine, and gives the caller back untouched:

```
*SWAP EDIT MYPROG.BAS
```

from EhBASIC edits a file and returns to your program, variables and screen as they were. One level deep; the thing being run must be an ordinary program, and a BASIC's far memory is not disturbed because nothing else touches it. BBC BASIC needs none of this: it lives on the co-processor, out of reach of anything running here.

SWAP -k is the same but *keeps* the screen the program left, instead of restoring the caller's. A file manager wants its display back; a script that ran a program to read what it printed wants the other thing, which is why RX uses this form — and why Microsoft BASIC runs every * command through it.

2.6 At the prompt

The line you are typing is editable: Left and Right move through it, Home and End go to its ends, Delete takes the character under the cursor and Backspace the one before it, characters insert where the cursor is, and Escape clears the line. It works across a line that has wrapped, and it takes accented letters.

Up and Down walk the last eight lines you typed, and Down past the newest gives you the empty line again. A line that repeats the one before it is not kept. The history lives in the ROM bank the line editor has to itself, so it survives a reset — but not a power cycle, which is what /STARTUP.BAT is for.

2.7 Aliases

ALIAS gives a name to a line.

```
ALIAS          list them
ALIAS LL DIR -1 define one
ALIAS LL       nothing after the name: remove it
```

Whatever you type after the alias is added to the end, so a definition takes arguments. Aliases are tried *last*, after every other rule, so one can never hide a real command: `ALIAS DIR ECHO no` is accepted and `DIR` still lists the directory. They live in memory — in a sideways bank, not in the 64 KB — so they survive a reset but not a power cycle, which is what `/STARTUP.BAT` is for.

2.8 The status bands

Turn on *status bands* (F7, under Terminal) and the console stops being the whole screen: a band at the top and a band at the bottom stay still while the text scrolls between them. Each band's height is a setting, and either may be nothing.

They are shared, and the split is worth knowing. **The top band is yours** — the machine draws the clock and the date in it, in whichever formats the Terminal page is set to, and on a laptop the battery: its charge, with an arrow up while it charges and down while it does not. **The bottom band is the running program's**, and a program that wants it writes there through `JIM; BANDS` in `/SYSTEM/BIN` is the demonstration. Programs that take the whole screen — the games, the editors — get the whole screen anyway: the bands are part of the console, and the console is what a program leaves behind when it asks for the glass.

2.9 The network

A URL is a file name. Anything the shell reads — `TYPE`, `LOAD`, `CP`, `RUN`, and the bare-word rule above — accepts `http://` or `https://` in place of a name, and the machine fetches it:

```
TYPE https://raw.githubusercontent.com/mlongval/k4510/master/README.md
CP http://example.org/game.prg game.prg
RUN http://example.org/game.prg
```

EhBASIC's LOAD and BBC BASIC's LOAD on the Tube take URLs the same way. The idea is the C64's Meatloaf cartridge, where a disk name could be an address on the internet; here it costs the ROM nothing, because the ROM never looks at a name — the host does the fetching. A typed line is 95 characters, and so is the longest URL.

Some URLs are more than a file: they are a place. TNFS is the little file protocol the FujiNet and Meatloaf servers speak, and ftp:// and sftp:// are what the rest of the world does; the machine's current directory can be on any of them:

```
CD tnfs://fujinet.online/
DIR
CD CBM
CD -
```

DIR lists the server, a bare name loads from it (so RUN, and the bareword rule, run programs straight off the internet), CD .. climbs, CD - comes home. The prompt shows where you are. A name and password go in the URL the ordinary way (sftp://me:secret@host/dir), and the FujiNet spelling — N:TNFS://... — is understood too.

Or give the place a name on the disk, and it stays there:

```
MOUNT tnfs://fujinet.online/ /MNT/FUJI
DIR /MNT/FUJI
MOUNT                                list what is mounted
UMOUNT /MNT/FUJI
```

Everything that reads a directory — DIR, CD, TYPE, RANGER, a program started by name — then works there as on the machine's own disk. A mounted place is read-only.

For programs that want a live connection there is the *N: device* at \$D900 (FujiNet's name for it): four channels, each a URL opened for reading and writing — tcp://host:port for a connection, http:// for a page. TELNET host port is the demonstration, a telnet.prg in /SYSTEM/BIN: what

you type goes out, what arrives is drawn by JIM, the terminal (Chapter 6), so a BBS gets its ANSI colours and CP437 art and the cursor and function keys go out as VT sequences; F12 hangs up (Escape belongs to the far end). It offers the far end the terminal types *xterm-color*, *VT220*, *VT100* and *ANSI*, and a Unix host that takes the first gets UTF-8 as well. The register map is in Chapter 15 and *core/net.h*.

So that there is no guessing, this is the whole list of what the machine speaks:

Scheme	What
http://, https://	fetch a file; a name anywhere a name goes; MOUNT shows one as a folder
tnfs://	a directory on a TNFS server
ftp://, sftp://	a directory on a server; sftp does not check the server's key
tcp://	a raw connection, through the N: device only

An *ssh session* is SSH (Chapter 13).

2.10 CAPSLOCK

CAPSLOCK toggles a caps-lock: while it is on, letters read from the keyboard come up uppercase, so a language whose keywords are uppercase — BBC BASIC, EhBASIC — needs no Shift held. Digits and symbols are untouched, so it is a caps lock and not a shift lock, and Shift gives you the *other* case while it is on. It is also suspended while a program is running, so a program that wants lower case gets it, and the lock comes back when the program does. CAPSLOCK ON and CAPSLOCK OFF set it explicitly; *CAPSLOCK works from inside a BASIC.

2.11 Scripts and the boot file

EXEC *name* runs a text file as shell commands, one per line. A line whose first character is # is a comment, and a blank line is ignored. At power-on the machine runs /STARTUP.BAT as such a script, if it exists — straight in, with no pause and nothing to catch. It is the place to put your aliases;

/SYSTEM/ETC/STARTUP.SAMPLE is one to copy from. If one ever stops the machine booting, page 28 is the way out.

A .BAT boots the machine and an .RX automates it: when a script needs to make a decision, read a result or talk to two things at once, that is RX's job (Chapter 12) and EXEC does not try to compete.

2.12 The monitor

MON (old-timers may type W0Z) opens the machine monitor at a * prompt; its grammar is Wozmon's, with 28-bit addresses:

```
*FF80.FF8F      examine a range
*0300:A9 41 60   store bytes
*0300R          run from $0300
*X              leave (or Q, or EXIT)
```

Anything else typed at the * prompt is a shell command, run on a clean machine and returned from. FILL from.to value and COPY from.to dest work on memory, by DMA, anywhere in the 256 MB.

The monitor is a program, MONITOR, and it loads at \$E000, in the RAM under the ROM, so that the memory it is there to look at — \$0800 to \$CFFF — is exactly as the last program left it. Addresses below \$10000 read as a program sees them: \$A000 to \$FEFF is that RAM, not the ROM, and the ROM shows only in its always-there page at \$FF00.

SUPERMON in /SYSTEM/BIN is the larger tool, in the tradition of Jim Butterfield's: M memory, D a disassembly that speaks the whole 45GS02, A an assembler, registers, hunting and filling — and @ escapes to the shell without leaving it.

2.13 Colours and palettes

COLOR fg bg sets the text colours (palette indices, in hex), and PALETTE the 256 colours behind them. The machine's own is the C64's sixteen, yellow on blue; /SYSTEM/ETC/PALETTES holds others:



```

K4510 Fantasy Computer - K/OS
CPU: 45GS10  RAM: 256 Mb
CHIPS: MELODY, VICKY, SHEILA, FRED, JIM

/HOME1 mon
monitor: addr  addr.addr  addr:b b b  addrR (28-bit hex)  X leaves
*ff80.ff8f
0000ff80: 4C 00 FF 4C 09 FF 4C 12 FF 4C 1B FF 4C 24 FF 4C
*
```

The monitor examining the system-call jump table. MON FF80.FF8F runs one line without entering the prompt.

PALETTE LOAD GREEN	a phosphor-green tube
PALETTE LOAD AMBER	an amber one
PALETTE RESET	the machine's own back

AMBER, GREEN and GREY are ramps — black and fifteen shades of one colour — so each carries a COLOR line that sets the text to a bright shade on black. Colour 1 in each is the brightest shade, because the shell draws its highlights in colour 1 (directories in DIR, the banner) and they must stay brighter than the text. C64 is the machine's own sixteen, as it boots them, and PEPT0 is the same sixteen as Philip Timmermann measured them off a real C64 — duller, warmer, and what one looked like on a television. A palette file is plain text — an index and three hex bytes to a line, # for a comment — and names only the entries it changes; PALETTE SAVE name writes one of yours. PALETTE LOAD in /STARTUP.BAT makes it the machine's.

2.14 Housekeeping

INFO is the machine's self-description; TIME the clock; MODE sets the text screen — CLS clears it and CLG clears the bitmap over it, whoever drew it — and both reach a BASIC through the * escape, which is how you tidy up after a demo that left its picture behind. HUSH silences the sound, whichever part of the machine is making it. BANNER clears the screen and prints the power-on banner again, which is a tidy way to end a session or start a screenshot. RESET restarts the machine from the shell, the same cold start the reset chord performs.

SETUP measures the computer the machine is running on, with the sound and the picture really running, and keeps the fastest clock it holds without a gap — for that computer, so a later boot there pays nothing. BENCH measures it again whenever you like: frames a second and gaps in the sound at every clock of the menu's ladder, about 25 seconds, the report in /SYSTEM/LOG/BENCH-NN.TXT. A clock is right for a host when it holds 60 frames a second with no gaps.

MODE on its own says where you are; MODE n moves. The second number is the margin: 1 (the default) keeps a one-cell gap at the top and the left, 0 uses every cell.

MODE	pixels	text	
0	640×480	80×60	the whole glass
1	640×240	80×30	the machine starts here
2	320×240	40×30	

The F7 menu, under Video, has the same two knobs: *Resolution* shows what the machine is actually in — it follows a MODE you type — and choosing another asks the ROM to perform it, because the console's geometry belongs to the ROM and not to the host. VICKY can draw smaller fields than these, but they are for a program that wants the pixels, not for a shell; a program asks for them through the video registers (Chapter 15).

The picture is always painted into 640×480: a 240-line mode has each line drawn twice, and a 320-wide one has its pixels doubled sideways. Raster lines and SHEILA's display list count the glass, 0–479, not the mode — which matters the moment a program asks VICKY for a 200-line field, where the first line of the picture is raster line 40.

2.15 When STARTUP.BAT is the problem

/STARTUP.BAT runs at power-on, and a bad one runs at every power-on. There is no moment to catch — the machine boots straight into it — so the way out is not a key held at the right instant but a switch that stays where you put it: the F7 menu, under Shell: *Run STARTUP.BAT*. That setting is the host's, kept in k4510.cfg, so it survives a power cycle and no wedged program in the machine can take it away from you. Boot clean, fix the file, switch it back on.

There is a second way, for one boot only:

```
./k4510 --no-startup.bat
```

(--no-startup does the same, as does K4510_NO_STARTUP=1 in the environment). It skips /STARTUP.BAT for that run and touches nothing: the F7 setting and k4510.cfg are left exactly as they were.

2.16 When something goes wrong

Type DUMP ON, make it go wrong again, then type BUG and answer its questions: the report it writes is the issue. Appendix B is the whole of it, on two pages.

Chapter 3

Every Command

This chapter is the whole vocabulary of the prompt, one line each, and it is not written by hand: `doc/guide/mkref.py` reads the shell's own command table out of the ROM's source, the programs out of `/SYSTEM/BIN` and the languages out of `/LANG` at the moment this book is built. A command the machine has and this list does not describe stops the build, and so does a description of a command that has gone — so what is below is what the machine you are reading about answers to. Chapter 2 is how the shell behaves; this is what you can say to it.

Each entry says where the word lives, which matters more than it looks:

ROM

in the resident part of the operating system, always there.

ROM, bank *n*

in one of the sideways banks (Chapter 14), paged in for the moment it runs.

runs MONITOR, runs TYPE

a word the ROM still answers to, which hands its line to a program on the disk. The work moved out of the ROM in September 2026 to make room; the word did not change.

/SYSTEM/BIN

a program. Programs run by name from anywhere, and take their arguments the same way the ROM's words do.

/LANG/*name*

a language: its folder, with its examples.

ROM, on the Linux

a word the ROM answers to by starting something on the Linux the machine runs on (Chapter 13).

Every word below also works from a BASIC, Forth or the monitor with a * in front, and a bare name that is not here is tried as a program, then as a CP/M program if F7 allows it, then as an alias, then as an RX script — in that order, so an alias never hides a command. Capitals are the convention; the shell does not mind either way.

Files and directories

CD [*dir* | *..* | *-* | *url*] *ROM, bank 3*

Change directory; the rest of the line is the name, spaces and all. Alone, to /. A *tnfs://* or *sftp://* URL goes there; *-* comes home. Also CHDIR.

CP [*-f*] **from to** *ROM*

Copy a file (a URL may be the source). Refuses to overwrite without *-f*. Not COPY, which is memory.

DIR [*-a*] [*-l*] [*dir* | *pattern*] *ROM*

List a directory. *-a* shows the hidden (dot) names, *-l* one to a line; a pattern matches here, * any run and ? any one character. Also LS.

LOAD *name* [*addr*] *ROM*

Load a file into memory, at *addr* or at the address in its header.

MKDIR *dir* *ROM, bank 1*

Make a directory.

MOUNT [*url* *path*] *ROM, bank 3*

Alone, list the mounts; otherwise show a *tnfs://*, *http://* or *https://* URL at a path of the disk.

RENAME [*-f*] *old new* *ROM*

Rename or move a file. Refuses to overwrite without *-f*. Also REN, MV.

RM [*-f*] *name* *ROM*

Move a file to /.TRASH, where DELETE *-r* gets it back. *-f* really removes it. Also ERASE, DEL.

RMDIR *dir* *ROM, bank 1*

Remove an empty directory.

SAVE *name from.to* *ROM, bank 1*

Write memory *from.to* to a file.

UMOUNT *path* *ROM, bank 3*

Take a mount away. Also UNMOUNT.

XD name*ROM, bank 1*

A file as a hex dump. Esc stops it. Also HEX.

Running things

ALIAS [name [text]]*ROM, bank 2*

List, define, or (name alone) remove an alias. Aliases are tried last, so none can hide a real command.

ECHO text*ROM*

Print the text.

EXEC name*ROM*

Run a text file as shell commands, one a line; # starts a comment. /STARTUP. BAT is run this way at power-on.

HELP*runs TYPE*

The command summary: TYPE /SYSTEM/ETC/HELP.

RUN [name | addr]*ROM*

Run a program; a bare name does the same. RUN addr jumps there.

SWAP [-k] command*ROM*

Put the whole 64 KB and the screen away, run the command on a clean machine, and give them back. -k keeps the screen the command left.

The screen

BANNER*ROM*

Clear the screen and print the power-on banner again.

CAPSLock [ON | OFF]*ROM, bank 1*

Toggle the caps lock: letters come up uppercase. Suspended while a program runs. Also CAPS.

CLG*ROM, bank 1*

Clear the bitmap over the text, whoever drew it.

CLS*ROM*

Clear the text screen.

COLOR fg [bg]*ROM, bank 1*

The text colours, as palette indices in hex. Also COLOUR.

- MODE [n [margin]]** *ROM, bank 1*
 Alone, say the mode; 0 640x480, 1 640x240, 2 320x240. The margin: 1 keeps a one-cell gap, 0 uses every cell.
- PALETTE [LOAD name | SAVE name | RESET | i rr gg bb]** *ROM, bank 2*
 The 256 colours: list them, set one, load a .PAL from /SYSTEM/ETC/PALETTES, save them, or put the machine's own back.

The machine

- DUMP [note | ON | OFF]** *ROM, bank 1*
 Write the machine's whole state to dumps/ on the host. ON writes one every fifteen seconds.
- HUSH** *ROM, bank 1*
 Silence the OPL2 and the sound sequencer.
- INFO [-v]** *ROM, bank 1*
 The machine's self-description, the clock in force included. -v names the exact build.
- RESET** *ROM*
 Cold-start the machine, as the reset chord does.
- TIME** *ROM, bank 1*
 The date and the time.

Memory

- COPY from.to dest** *runs MONITOR*
 Copy memory, by DMA. Not files: that is CP.
- FILL from.to value** *runs MONITOR*
 Fill memory with a byte, by DMA.
- MON [line]** *runs MONITOR*
 The machine monitor, Wozmon's grammar with 28-bit addresses. With a line, runs it and returns. Also WOZ.

Languages

BBC	<i>ROM</i>
BBC BASIC on the Tube (Chapter 6). Also BBC.	
CPM [command]	<i>ROM</i>
CP/M 2.2 on the Z80 (Chapter 9). A command runs at boot.	
EHBASIC	<i>/LANG/EHBASIC</i>
Enhanced BASIC with the machine's graphics (Chapter 4).	
FORTH	<i>/LANG/FORTH</i>
Tali Forth 2 (Chapter 7).	
LOGO	<i>/LANG/LOGO</i>
Turtle graphics (Chapter 8).	
MSBASIC	<i>/LANG/MSBASIC</i>
Microsoft's 6502 BASIC of 1977 (Chapter 5).	
RX name	<i>/LANG/RX</i>
Run a REXX script (Chapter 12); a bare HELLO runs HELLO.RX.	

The Linux beneath

!command or !	<i>ROM, on the Linux</i>
Run a command on the Linux beneath, or (alone) open a shell there. Can be locked off.	
CC name	<i>ROM, on the Linux</i>
Compile name.C here with cc65 into name.prg.	
PAS name	<i>ROM, on the Linux</i>
Compile name.PAS here with Mad Pascal into name.prg.	
SSH [user@]host	<i>ROM, on the Linux</i>
An ssh session, through the Linux's ssh. Locked off with !.	

Programs in /SYSTEM/BIN

BANDS	<i>/SYSTEM/BIN</i>
A program writing to the bottom status band.	

BENCH	<i>/SYSTEM/BIN</i>
About 25 s: frames per second and sound gaps at every clock step, to /SYSTEM/LOG/BENCH-NN.TXT.	
BOOK [n word page]	<i>/SYSTEM/BIN</i>
This handbook, on the machine: the contents, chapter n, or the chapter whose title holds the word. Tab chooses a link, Enter follows it, Backspace comes back.	
BUG	<i>/SYSTEM/BIN</i>
Asks seven questions about a fault and writes the report (Appendix B).	
CHROUT	<i>/SYSTEM/BIN</i>
How fast the ROM's console prints.	
DELETE [-l -r name -e name]	<i>/SYSTEM/BIN</i>
The trash: list it, put a file back, empty it, or send a file there.	
EDIT [name]	<i>/SYSTEM/BIN</i>
The modeless editor (Chapter 11).	
KEYTEST	<i>/SYSTEM/BIN</i>
Asks for every key and checks what arrives.	
KOMMANDER	<i>/SYSTEM/BIN</i>
The two-panel file manager.	
MONITOR	<i>/SYSTEM/BIN</i>
The monitor as a program: MON, WOZ, FILL and COPY run it.	
MOUSETEST	<i>/SYSTEM/BIN</i>
The mouse registers, live, with a sprite pointer.	
PADTEST	<i>/SYSTEM/BIN</i>
The held-keys register, live: a gamepad's first test.	
PETSCII	<i>/SYSTEM/BIN</i>
JIM, the terminal, speaking PETSCII.	
RANGER	<i>/SYSTEM/BIN</i>
The miller-column file manager.	
SAY text	<i>/SYSTEM/BIN</i>
Prints its arguments: the smallest program there is.	
SETUP	<i>/SYSTEM/BIN</i>
Measures this host thoroughly and keeps the clock it settles on.	
SPLIT	<i>/SYSTEM/BIN</i>
A split screen held by SHEILA: blitter lines above, four rows of text below, and how many lines a second.	

SUPERMON	<i>/SYSTEM/BIN</i>
Jim Butterfield's monitor, grown up: an assembler and a 45GS02 disassembler.	
TELNET host [port]	<i>/SYSTEM/BIN</i>
A terminal on a TCP connection. F12 hangs up.	
TYPE name	<i>/SYSTEM/BIN</i>
A file, a screenful at a time; Esc or Q stops. A URL works.	
VI [name]	<i>/SYSTEM/BIN</i>
The modal editor (Chapter 11).	

Chapter 4

EhBASIC

The machine has two BASICs, and this is the one with the machine in it: EhBASIC 2.22 — Lee Davison's Enhanced BASIC — with this machine's additions: graphics statements that ride the blitter, floating point on the MATH unit, and an escape hatch to the shell. It has no sound *keyword*: a program that wants a note pokes the sound sequencer at \$D5E0 or the OPL2 at \$D480 directly, which on a machine with a friendly memory map is not much of a hardship (Chapter 15). The other BASIC, Microsoft's own, has none of this and is proud of it (Chapter 5).

```
RUN EHBASIC
```

It comes up ready: the banner, 47103 Bytes free — more than twenty-one C64s more than a C64 — and a Ready prompt — the traditional Memory size ? question is gone; the machine knows.

4.1 Ten minutes of it

```
RUN "DEMOS.BAS"
```

The menu chains: RUN "name" (or LOAD inside a running program) loads another BASIC program and runs it — that is how one BASIC program hands over to another, exactly as on a C64.

4.2 The machine's own statements

```
GRAPHICS 2          640x480 bitmap over the text
PLOT X,Y,C          LINE X1,Y1,X2,Y2,C
TRI X1,Y1,X2,Y2,X3,Y3,C
PALETTE I,R,G,B     GCLS          GRAPHICS 0
```

```

K4510  EHBASIC DEMOS

1  LINES      SPINNING LINES (BLITTER LINE OP)
2  TRIS       300 RANDOM TRIANGLES
3  SINE       SINE WAVES (MATH UNIT)
4  STARS      A STARFIELD (ANY KEY LEAVES)
5  README     HOW TO USE EHBASIC HERE
I  INVADERS   SPACE INVADERS ON THE TEXT SCREEN
J  INVADER2   SPACE INVADERS WITH HARDWARE SPRITES
G  GRAPH2D    PLOT Y=F(X), PAN AND ZOOM
H  GRAPH3D    A SURFACE IN WIREFRAME, ROTATE IT
B  BENCHMARKS MENU
*  K/OS COMMANDS: *DIR, *CD /LANG/EHBASIC, *TYPE NAME, *INFO ... (@ WORKS TOO)
Q  QUIT TO READY

CHOICE?

```

The demo menu. A key picks an entry; each demo returns here when it ends.

Colours 0–15 belong to the text screen; demos use 16 and up. GRAPHICS 0 puts the text mode and its palette back, whatever the program changed.

4.3 Hardware sprites

The 128 sprites VICKY carries are reachable from BASIC, which is unusual enough to be worth saying plainly: this is not a bitmap being redrawn, it is the video chip carrying the picture for you.

```

SPRDEF n,page,w,h,bpp  give sprite n a shape
SPRITE n,x,y           put it there and show it
SPROFF n               hide it

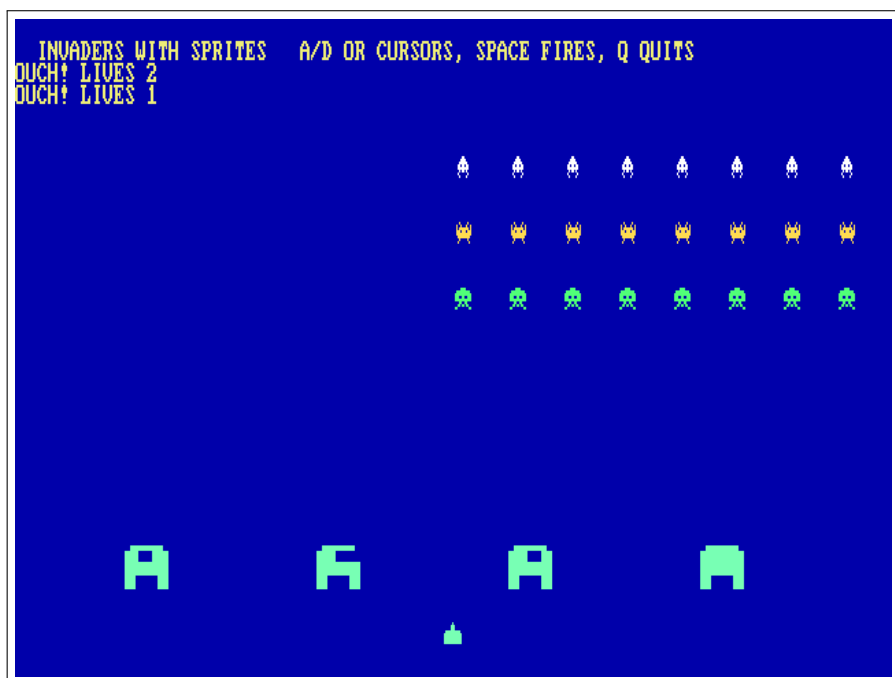
```

SPRDEF says where the pixels are and what shape they make: *page* is the 256-byte page the data starts at, which is how a 16-bit BASIC

number reaches into 256 MB (page*256 is the address); *w* and *h* are 8, 16, 32 or 64; *bpp* is 4 or 8. `SPRITE` positions the sprite and turns it on, `SPROFF` hides it. Colour 0 is transparent, and there is no per-line limit — all 128 may be on the same raster line.

Each statement re-points the chip at the attribute table and re-enables sprites, so a `GRAPHICS 0` or a shell `MODE` in between costs you nothing: the next sprite statement puts them back.

`INVADER2.BAS` in `/LANG/EHBASIC/EX` is the demonstration — Space Invaders with the arcade shapes, poked 4 bits-per-pixel into \$040000 through a bank register, four bases that erode as they are hit, and thirty-one sprites moving. `INVADERS.BAS` beside it is the same game on the text screen, which is a fair way to see what the sprites buy you.



`INVADER2.BAS`: twenty-four invaders, four bases and a cannon, every one of them a hardware sprite. The bases erode because their shape pages are poked, not redrawn.

4.4 The * escape

Any shell command works from BASIC with * in front — *DIR, *CD EHBASIC, *MON, *DUMP a note — and *BYE leaves BASIC for the shell. Escape or Ctrl-C stops a running program (and silences the sound); the program's variables survive for PRINT-style post-mortems.

4.5 Editing the program in VI

*VI and *EDIT, with nothing after them, edit the program that is in memory:

```
10 PRINT "HELLO"  
*VI
```

BASIC writes the program to a temp file, opens the editor on it, and reads it back when you leave — so what you type in the editor is what you LIST afterwards. Give either one a file name and nothing special happens: *VI notes.txt is the ordinary * escape, and the editor edits that file.

The trip out and back is not free. The editors load where the interpreter lives, so the shell command underneath is SWAP, which puts all 64 KB and the screen aside first and restores them afterwards. And because the program is written out and read back, *variables do not survive* — this is an immediate-mode thing, like LOAD. The temp file (EDITTMP.BAS) is left behind, which is occasionally a useful accident.

Why the first line is blank: SAVE starts its output with a newline, so the editor opens on an empty line 1 with the program below it. Harmless — BASIC ignores it on the way back in — but it is why the line count is one more than you expect.

Chapter 5

Microsoft BASIC, 1977

EhBASIC is the BASIC with the machine in it. This is the other one, and it is the real thing:

```
MSBASIC
```

Microsoft's 6502 BASIC — not an imitation of it, but the source Microsoft released in 2025, assembled in a *pure* configuration: the newest base in the tree, nine-digit floating point, every fix through revision 2C, and none of the additions any particular manufacturer made to it. It is the interpreter that became Commodore's and Apple's before either of them touched it, and it is here because a fantasy machine of the mid-eighties would have been sold one. Not a byte of Microsoft's source is changed: everything the machine adds is caught a level below the interpreter, where the keyboard and the screen come in, so the BASIC never knows.

It loads at \$7000, comes up with its own banner, and gives you its famous prompt: OK. It answers the MEMORY SIZE? question itself — a pure Microsoft cold start asks where memory ends and, given no answer, finds out by walking upwards, straight through its own image — so you start with a program area of 26 KB.

5.1 Saving and loading

Microsoft left SAVE and LOAD to each manufacturer, and every one of them wrote their own. These are the K4510's:

```
SAVE "HELLO"  
LOAD "HELLO"
```

A program is kept as *text*: HELLO.BAS is its LIST, word for word, in the directory you are standing in. .BAS is added to a name without a dot.

```

CPU: 45GS10  RAM: 256 MB
CHIPS: MELODY, VICKY, SHEILA, FRED, JIM

/HOME1 msbasic
MICROSOFT BASIC ON THE K4510
*HELP FOR K:OS COMMANDS, *BYE TO LEAVE

MEMORY SIZE? 28672
TERMINAL WIDTH? 80

26623 BYTES FREE

COPYRIGHT 1977 BY MICROSOFT CO.

OK
10 FOR I=1 TO 6
20 PRINT I, I*I, SQR(I)
30 NEXT I
RUN
 1          1          1
 2          4          1.41421356
 3          9          1.73205081
 4         16           2
 5         25          2.23606798
 6         36          2.44948974

OK

```

Microsoft BASIC: its own banner, the memory the machine gave it, and a program run.

That makes the file something every other program on the machine can read — TYPE shows it, VI edits it, you can write one on the host — and loading it is exactly as if it were being typed at the keyboard, only faster and without the echo. SAVE and LOAD with no name use the last name you gave, which is PROGRAM.BAS until you have given one. LOAD of a file that is not there says ?FILE NOT FOUND and leaves the program you had alone; SAVE inside a running program ends it, as LIST does.

5.2 The * escape

A line beginning with * typed at the OK prompt is not BASIC's but K/OS's: *DIR, *CD, *TYPE, any command in Chapter 3. A program started this way runs on a clean machine and gives BASIC back as it was — the command underneath is SWAP -k (Chapter 2). A * typed at an INPUT prompt is data,

not a command.

Three words are BASIC's own:

***BYE (or *QUIT)**

hands the machine back to the shell, its directory and its screen intact. Microsoft's BASIC has no way out of its own, so this one is built underneath it.

***VI**

edits the program in memory: it is saved under its name, VI opens on it, and when you leave VI it is loaded back. So what you type in the editor is what you LIST afterwards. Variables do not survive the round trip, as with any LOAD; if the save fails, nothing else happens. *VI name with a name is the ordinary shell command, and edits that file.

***HELP**

says the two things above, then prints the shell's own command summary.

5.3 What it does not have

Sound, graphics, sprites: none, and it is not going to grow any. Adding keywords means editing Microsoft's interpreter, which is the one thing this port does not do. A program that wants a colour or a note can POKE the chips directly (Chapter 15); for anything more, EhBASIC is next door.

The arrow keys do not edit the line you are typing: Microsoft's line input of 1977 had no cursor to move. They are ignored there, so an arrow typed by habit leaves no stray character, and a program that reads the keyboard with GET still receives them.

Chapter 6

The Tube: BBC BASIC

The BBC Micro's most elegant idea was the Tube: a fast port through which a *second processor* — another CPU with its own memory — could take over the computation while the Beeb kept the keyboard, the screen and the discs. The K4510 has a Tube of its own at \$0800, and the first thing fitted to it is Richard Russell's BBC BASIC, running on the host machine with a flat 256 MB of its own.

```
BBC
```

You get the > prompt of a BASIC with real power behind it, and it is *fast* — the co-processor is not cycle-matched to 1985:

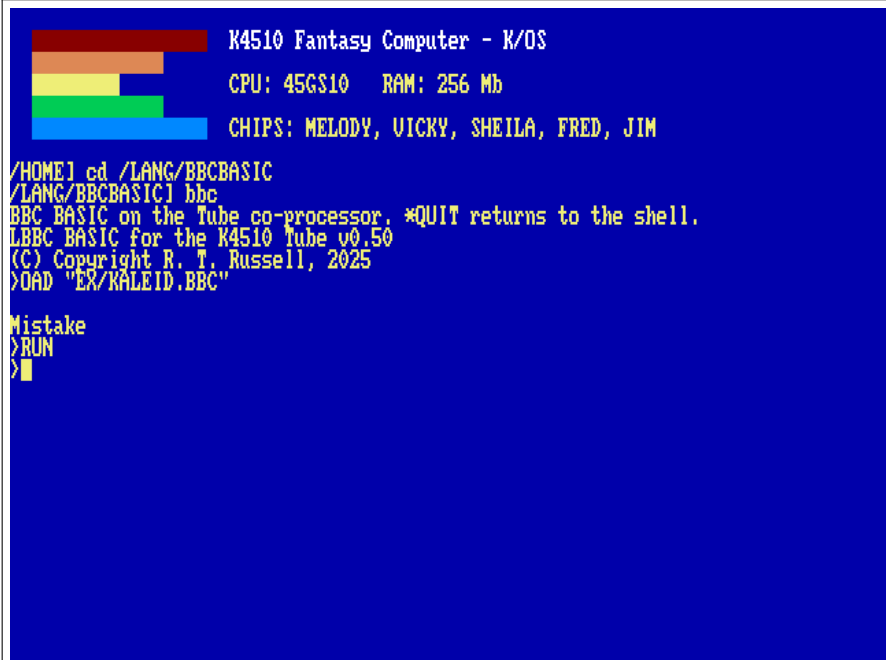
```
HIMEM=PAGE+250*1024*1024  
DIM space 200*1024*1024
```

both just work. Type *QUIT to hand the console back to the shell.

6.1 Graphics and sound

The console cannot show what it cannot see, so the Tube speaks up: MODE, GCOL, PLOT, MOVE, DRAW, CIRCLE and the palette arrive at the machine as escape sequences, and the Tube's gatekeeper (the *Tube ULA*) executes them on the VICKY blitter. SOUND and ENVELOPE-less music go the same way, onto the machine's four-channel sound sequencer and out through the OPL2 — the classic Beeb SOUND chan,amp,pitch,dur, queued per channel like the original. Channel 0, which was the Beeb's noise, is a feedback-heavy FM patch; 1 to 3 are plain two-operator tones. The patch is written again on every note, so a program that has zeroed the chip for its own purposes does not silence the sequencer.

```
CD /LANG/BBCBASIC
BBC
LOAD "EX/KALEID.BBC"
RUN
```



The screenshot shows a BBC BASIC terminal window with a blue background. At the top, there is a graphic consisting of four horizontal bars of different colors: red, orange, yellow, and green. To the right of these bars, the text reads: "K4510 Fantasy Computer - K/OS", "CPU: 45GS10 RAM: 256 Mb", and "CHIPS: MELODY, VICKY, SHEILA, FRED, JIM". Below this, the terminal shows the following commands and output:

```
/HOME1 cd /LANG/BBCBASIC
/LANG/BBCBASIC1 bbc
BBC BASIC on the Tube co-processor, *QUIT returns to the shell.
LBBC BASIC for the K4510 Tube v0.50
(C) Copyright R. T. Russell, 2025
>OAD "EX/KALEID.BBC"

Mistake
>RUN
>■
```

KALEID.BBC: MODE 2 kaleidoscope, drawn by BBC BASIC on the Tube, painted by VICKY.

The /LANG/BBCBASIC/EX directory carries a period programme selection: KALEID, CIRCLES, ROSES, MOUNTAIN, BOUNCE, CLOCK (a live analogue clock face) and TUNE — Frère Jacques as a three-voice round, the sound sequencer's party piece.

6.2 Sprites

VICKY has 128 hardware sprites, and BBC BASIC reaches them the way RISC OS reached its own: VDU 23,27 selects and shapes, PLOT &ED places.

There is no sprite file. A sprite is *captured* from the bitmap after BBC BASIC has drawn it with the words it already knows:

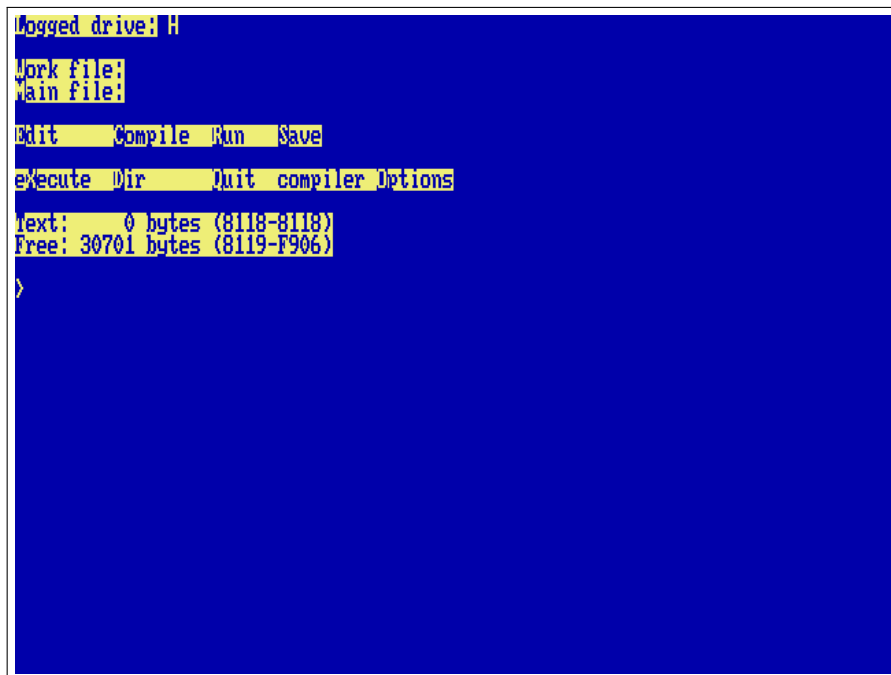
```
MODE 2
GCOL 0,1:CIRCLE FILL 40,40,30
MOVE 8,6:VDU 23,27,1,0,32,32|
CLG
PLOT &ED,600,500
```

VDU 23,27,0, <i>n</i>	select sprite <i>n</i> (0–127) for PLOT
VDU 23,27,1, <i>n,w,h</i>	capture <i>n</i> , $w \times h$ pixels (8/16/32/64), bottom-left at the graphics cursor
VDU 23,27,2, <i>n</i>	hide <i>n</i>
VDU 23,27,3, <i>n,f</i>	flip: bit 0 horizontal, bit 1 vertical
VDU 23,27,4, <i>n,z</i>	depth: drawn after layer <i>z</i> (default 1, over the bitmap)
VDU 23,27,5, <i>n,m</i>	sprite <i>n</i> shows sprite <i>m</i> 's picture
PLOT &ED, <i>x,y</i>	show the selected sprite, bottom-left at <i>x,y</i>

A placed sprite is a register write: moving sixty-four of them every frame costs the interpreter nothing but the PLOTs. `SPRITES.BBC`, `INVADERS.BBC` and `TUNNEL.BBC` beside them are the demonstrations (the last one moves nothing at all — it cycles the palette with VDU 19).

6.3 The terminal: JIM

The console you see when the Tube is running is not the ROM's own terminal but *JIM*, at \$DA00 — the Beeb's third I/O page, given a job here. JIM is a VT100 with the ANSI colours and the VT220 editing sequences (insert and delete character and line, erase character, scroll regions, origin mode, DEC line drawing, cursor-position and identity reports), built into the machine the way a real 8-bit computer got a serious terminal: as a card, not a program. It draws on the same text screen as the ROM console, inside the window the ROM gives it, so the two share one screen and one cursor. Anything that needs a terminal writes its byte stream to \$DA00 and reads its answers from \$DA02; keys pushed through \$DA03 come back as the VT sequences the far end expects (arrows, Home/End, PgUp/PgDn, F1–F12, Delete as \$7F). The register map is in `core/term.h`.



Turbo Pascal 3 (H: user 3) on JIM: the menu bar in reverse video, the compiler's own screen handling on a VT100 that is a chip.

For CP/M this settles the question every program asks at install time: tell WordStar's WSCCHANGE, Turbo Pascal's TINST, ZDE and the rest that the terminal is a *VT100* (or *ANSI*: the same sequences). Turbo Pascal 3 on H: user 3 comes up with its menu already right; WordStar 4 on E: wants one pass of WSCCHANGE. BBC BASIC's console edition speaks the same language natively, so COLOUR, CLS and PRINT TAB(*x*,*y*) land where they should. Bytes \$80–\$FF are drawn as CP437 glyphs, which is what BBS ANSI art is made of.

6.4 Star commands

A line starting with * goes to the machine: *DIR, *CD, any shell command — and BBC BASIC's own file words (LOAD, SAVE) read and write the machine's filesystem directly, because the co-processor lives inside fs/ too.

The Tube starts in the shell's current directory, so `CD /LANG/BBCBASIC` then BBC lets `LOAD "EX/KALEID.BBC"` work without spelling the whole path; the machine's filesystem is the co-processor's whole world — `fs/` is shown as `/`, the host tree above it hidden, and `*CD ..` stops at the root.

Edges, honestly: `POINT(` and `TINT` are not implemented; `GCOL` modes 1–4 draw plain; `VDU 5` text prints as text.

Chapter 7

Forth

The machine's third language is native: no Tube, no co-processor, just 45GS10 machine code loaded at \$8C00, with the dictionary below it. It is Tali Forth 2 — a public-domain, ANS-flavoured Forth written for the 65C02, which this CPU speaks as a strict superset.

```
FORTH
```

```
2 3 + . 5 ok
: cube dup dup * * ; ok
7 cube . 343 ok
```

7.1 The machine at your fingertips

Forth's oldest habit is poking hardware, and this machine is one large, friendly memory map. C@ and C! reach every register of Chapter 15 directly:

```
hex
D000 C@ .      read a VICKY register
D5E0 80 swap C!  silence the sound sequencer by hand
```

7.2 The assembler

Tali brings an interactive 65C02 assembler and a disassembler, kept in because on this machine they are the point:

```
' dup 10 disasm      disassemble a word of the kernel
assembler-wordlist >order
here 2 lda.# 0 sta.z drop
```

```

K4510 Fantasy Computer - R/OS
CPU: 45GS10  RAM: 256 Mb
CHIPS: MELODY, VICKY, SHEILA, FRED, JIM

/HOME1 forth
Tali Forth 2 on the K4510 (native 45GS10)

Tali Forth 2 for the 65c02
Version 1.1 06. Apr 2024
Copyright 2014-2024 Scot W. Stevenson, Sam Colwell, Patrick Surry
Tali Forth 2 comes with absolutely NO WARRANTY
Type 'bye' to exit
2 3 + . 5 ok
: cube dup dup * * ; ok
7 cube . 343 ok
' dup 10 disasm
37748 36419 jsr 1 STACK DEPTH CHECK
37751 dex
37752 dex
37753 2 lda.zx
37755 0 sta.zx
37757 3 lda.zx
ok

```

A Forth session: the banner, arithmetic, a new word, and the kernel disassembling itself.

DISASM is strictly 65C02: at a 45GS10-only opcode it prints a polite ? and moves on.

7.3 Manners

The dictionary has about 31.7 KB free for your words. BYE returns to the shell with the screen and the session exactly as you left them.

No file words yet: Forth source lives in /LANG/FORTH but must be typed. An INCLUDED that reads through the machine's storage device is on the list.

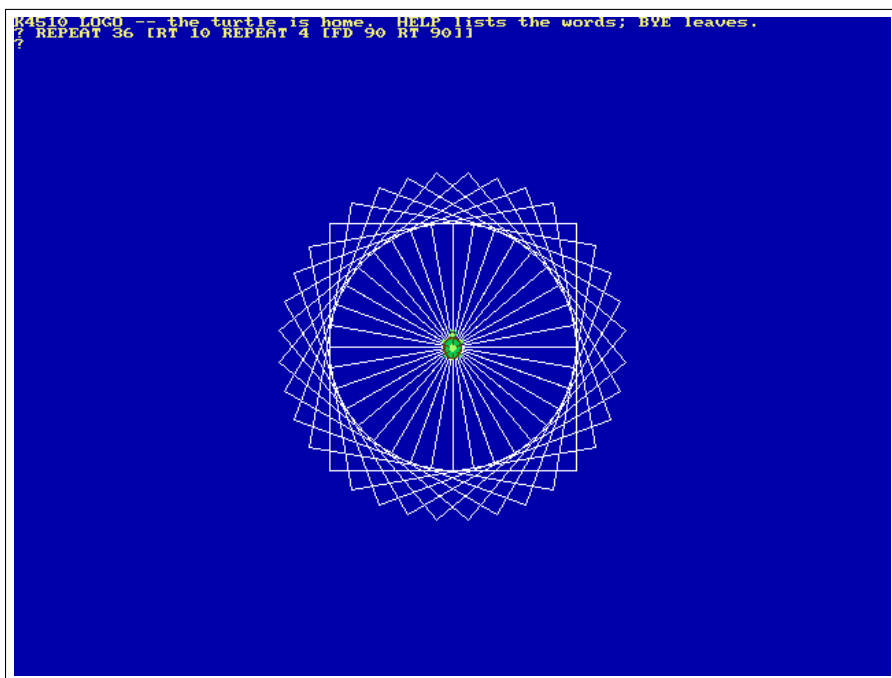
Chapter 8

LOGO

LOGO is the language that taught a generation of children that a computer does what it is told, and it did it with a turtle: a small arrow on the screen that walks forward, turns, and leaves a line behind it. The K4510's is new, written for this machine in September 2026, and native — 45GS10 code, no co-processor.

```
LOGO
FD 100 RT 90
REPEAT 4 [FD 100 RT 90]
```

The picture is VICKY's 640×480 bitmap *under* the console, which is the classic LOGO screen: the turtle draws below and the prompt lives above, so you can see what you asked for and what you are asking next at once. The turtle itself is a small green turtle, a hardware sprite in sixteen headings — one every 22.5 degrees, drawn once and turned by `tools/mkturtle.py` when the machine is built, so turning it is a matter of pointing the sprite at the right picture. LOGO puts the console into 640×480 for the session, as EhBASIC's GRAPHICS 2 does, and back into the mode it found at BYE. Every line is the blitter's, and every number is a real one — IEEE floating point, done by the MATH unit, so a heading is an angle and SQRT 2 is what it should be.



LOGO: a square repeated round a circle, drawn by the blitter under the console's text.

8.1 The words

FD n, BK n	forward, back
RT n, LT n	right, left, in degrees
PU, PD	pen up, pen down
HT, ST	hide, show the turtle
HOME, CS	to the centre; clear the screen as well
SETXY x y	put it somewhere
SETX, SETY, SETH	one coordinate, or the heading
SETPC n	the pen's colour
FILL	paint the area under the turtle
XCOR, YCOR, HEADING	where it is
REPEAT n [...]	do it n times; REPCOUNT says which time this is
IF c [...]	decide
IFELSE c [...] [...]	decide between two
MAKE "x expr, :x	a variable, and its value
PRINT, SHOW	print a value
RANDOM, SQRT, SIN, COS, INT, ABS	numbers
TO name :a ...END	a procedure; STOP and OUTPUT leave it
LOAD "name	run a .LGO file
EDIT "name	edit one in VI, then run it
HELP, BYE	the words; back to K/OS

8.2 Procedures

A procedure is a new word, and it may call itself:

```
TO TREE :N
  IF :N < 5 [STOP]
  FD :N LT 30 TREE :N * 0.7
  RT 60 TREE :N * 0.7
  LT 30 BK :N
END
TREE 80
```

A procedure's inputs are its own; MAKE writes the nearest variable of that name, or makes a global one. Recursion is real and goes deep — deep enough for any tree — and a procedure that recurses for ever says so

rather than taking the machine with it.

8.3 Files

LOAD "TREE runs TREE.LGO from the directory you are in, or from the examples in /LANG/LOGO/EX: SQUARE, SPIRAL, TREE and SNOW. EDIT "MINE opens MINE.LGO in VI (Chapter 11) and runs it when you leave the editor, which is the way to write anything longer than a line. A .LGO file is plain text, one line as you would type it after another.

Chapter 9

CP/M: the Z80 Second Processor

In 1984 Acorn sold a Z80 Second Processor for the BBC Micro; its purpose was to run CP/M, the operating system that owned business computing before the IBM PC. It sat on the Tube. So does ours. CPM at the shell boots CP/M 2.2 on a Z80 co-processor (RunCPM, on the host), and the A0> prompt that defined a decade appears. The boot banner cheerfully measures the Z80 in *gigahertz* — read that number knowingly: it is the emulated Z80’s speed *on whatever host you run the emulator on* (a laptop, in this book’s screenshots), so it varies machine to machine. For scale, the real thing shipped at 4 MHz.

CPM

9.1 Drives are folders

Drives A: to P: are the host folders `fs/CPM/A` through `fs/CPM/P`; CP/M’s user areas 0–15 are numbered subfolders. Drop any `.COM` file into `fs/CPM/A/0` and run it by name. `DIR`, `TYPE`, `ERA`, `REN`, `SAVE` and `USER` are built into the CCP; `EXIT` hands the console back to the K/OS shell. `Ctrl-C` stops a running CP/M program that is listening for it, as it did in 1978 — an MBASIC loop, say.

Three of the drives are shortcuts rather than folders of their own: K: is the machine’s own filesystem, so CP/M and K/OS can hand files to each other in both directions; P: is Turbo Pascal and D: is WordStar, each pointing straight at the user area the program lives in.

And one drive letter is spoken for: N: is the network, reserved and empty. It is one letter and not two because on this machine the scheme lives in the *name* rather than in the device — FujiNet and Meatloaf are two URL schemes over one namespace, not two kinds of place — so N: will be a mount point showing whatever the shell’s `CD tnfs://` has the

```

A: ED      COM | EXIT   COM | EXIT   SUB | EXIT   Z80
A: FORMAT  COM | FORMAT  SUB | FORMAT  Z80 | INFO   COM
A: INFO    SUB | INFO    TXT | INFO    Z80 | K-MBAS SUB
A: K-TURBO SUB | K-WS    SUB | LOAD    COM | LST     TXT
A: LU      COM | MAC     COM | MAKEFCB LIB | MBASIC  COM
A: MDIR    COM | MLOAD   ASM | MLOAD   COM | MLOAD   DOC
A: MOVCPM  COM | OPCODES DOC | PIP     COM | README  TXT
A: RSTAT   COM | RSTAT   SUB | RSTAT   Z80 | RUNUT   BAS
A: SLRASM  COM | STAT    COM | SUBMIT   COM | SUBMITD COM
A: SYSGEN  COM | TE      COM | UNARC    COM | UNCR    COM
A: UNZIP   COM | USQ     COM | XMODEM   COM | XSUB    COM
A: Z2HDR   LIB | Z3BASE  LIB | Z3HDR   LIB | Z80ASM  COM
A: Z80ASM  PDF | Z80CCP  ASM | Z80CCP  SUB | ZCPR2   ASM
A: ZCPR2   SUB | ZCPR3   ASM | ZCPR3   SUB | ZEX     COM
A: ZEX     Z80 | ZEXALL  COM | ZEXDOC  COM | ZSID    COM
A: ZTRAN   COM
A0>type readme.txt
CP/M 2.2 ON THE K4510
-----
You are on the Z80 second processor (a 3 GHz Z80, as it happens),
reached over the Tube. Acorn sold exactly this in 1984.

Drives A: to P: are the folders fs/CPM/A ... fs/CPM/P on the host;
user areas 0-15 are subfolders. Drop .COM files in and run them.
N: is reserved for the network -- see N:README.TXT.

DIR, TYPE, ERA, REN, SAVE and USER are built into the CCP.
EXIT returns to the K/OS shell.

A0>

```

CP/M 2.2 over the Tube: the CCP's DIR and TYPE, on drive A.

machine looking at. The plumbing is not written; the letter is held so that nothing else takes it.

9.2 Starting a program from the shell

CPM on its own gives you the A0> prompt. CPM command runs a command at boot instead, through the CCP's AUTOEXEC.TXT. One line is all the CCP reads — but a name with no .COM to match runs the .SUB of that name, and that may be as long as you like. /CPM/A/0/K-TURBO.SUB is the pattern:

```

P:
TURBO
EXIT

```

Ending in EXIT is what makes the round trip whole: quitting a CP/M program only drops you to the CCP, and the EXIT carries you

the rest of the way back to the K/OS prompt. With that in place ALIAS TURBO CPM K-TURBO makes TURBO a word you can type at the shell; /SYSTEM/ETC/STARTUP.SAMPLE defines that one, WS and MBASIC.

Why not USER in a submit. The obvious script is H:, USER 3, TURBO — and it dies on the second line. The submit in progress is a file, \$\$\$SUB, and it lives on A: user 0; USER 3 walks away from it and it cannot be read any more. That is CP/M's own behaviour and not RunCPM's doing. Pointing a drive at the user area, as P: and D: do, keeps USER out of it.

9.3 Typing a CP/M program's name directly

The shell can be told to treat an unknown word as a CP/M program: F7 → Shell → *CP/M.COM by name*. With it on, STAT at the /] prompt starts CP/M, runs STAT.COM from A: and comes back.

It is off to begin with, and deliberately so — D typed for DIR should not start a Z80 program. It changes only the guess: a command naming CPM outright, an alias among them, works either way. There is nothing in a .prg to tell the two apart, incidentally — its header is a load address and a run address, and a .COM begins with Z80 code that would read as a perfectly plausible one — so the extension is what decides.

9.4 Software

The RunCPM project ships a complete system disk (DISK/A0.zip in its repository): Digital Research's own ASM, MAC, DDT, ZSID, STAT, PIP and ED, the SUBMIT batch system, Microsoft BASIC, a Z80 assembler with its manual, XMODEM, and the source code of the BDOS and CCP — buildable on the machine itself. One `unzip A0.zip -d fs/CPM/` installs the lot. Beyond that lies the whole surviving CP/M software world: WordStar, Turbo Pascal, dBase II, Zork, one archive away.

Edges, honestly: line-mode programs (assemblers, compilers, MBASIC, adventures) work today, and so does the full-screen soft-

ware: the console is JIM, a VT100 in hardware (Chapter 6). WordStar 4 on E: user 3 comes installed for “ANSI Standard” at 79×29 — WS and you are editing; Turbo Pascal 3 on H: user 3 needs nothing either. A program that asks its terminal type at install time wants “VT100” or “ANSI”.

The four arrow keys work in CP/M software: while the Tube is running CP/M the machine sends them down as the WordStar keys — Ctrl-E, X, S and D for up, down, left and right — which is what the programs of 1984 read, WordStar and Turbo Pascal (Chapter 10) among them. The rest of the navigation keys do not: Home, End, PgUp and PgDn are two-key ^Q sequences in WordStar, and there is no single key the machine could honestly send for them, so type those as the program’s manual says.

```

E:README.ME          POI LOI COI Insert Align          Large-File
          EDIT MENU
CURSOR      SCROLL    ERASE      OTHER      MENUS
^E up        ^W up      ^G char    ^J help    ^O onscreen format
^X down      ^Z down    ^T word    ^I tab     ^K block & save
^S left      ^R up screen ^V line    ^U turn insert off ^P print controls
^D right      ^C down    Del char   ^B align paragraph ^Q quick functions
^A word left  screen    ^U unerase ^N split the line  Esc shorthand
^F word right

L-----!-----!-----!-----!-----!-----R
                        --THE README FILE--
                        -----
README contains late-breaking news and tips about WordStar,
and information about printers.

THE DISKS THAT CAME IN YOUR PACKAGE
-----

The file HOMONYMS.TXT is included on the Speller disk
contrary to what is listed in Appendix D.

INSTALLATION
-----

WINSTALL and WSCCHANGE
  
```

WordStar 4 editing its own README.ME: the edit menu, the ruler, the text — 79×29 on JIM.

Chapter 10

Pascal

The machine has two Pascals, and they share nothing but the language. Keep them apart in your head from the start:

	Turbo Pascal 3	Mad Pascal
What it is	a program of 1985	a cross-compiler of today
Where it runs	on the machine: the Z80, under CP/M	on your desktop
What it makes	a CP/M .COM, for the Z80	a K/OS .prg, for the 45GS10
What it can touch	CP/M's world only	every chip in the machine
How you get it	you supply it; it is Borland's	in the repository, free

Nothing carries over between them — not the source, not the toolchain, not the programs they produce. The first section is one, the second is the other.

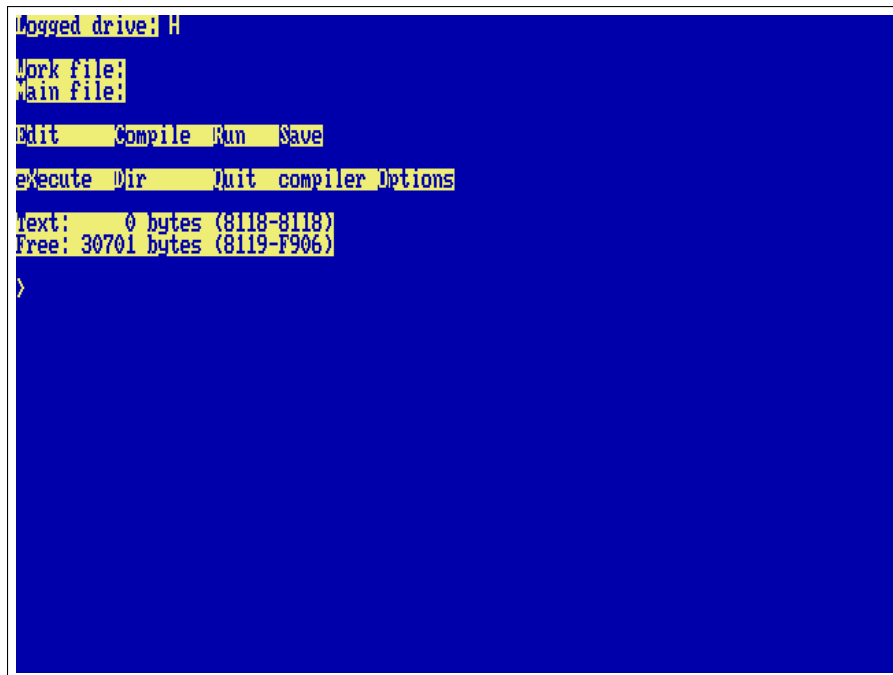
10.1 Turbo Pascal 3, on CP/M

Borland's Turbo Pascal 3 is a CP/M program, so it runs where CP/M runs: on the Tube's Z80 (Chapter 9), in its own world, exactly as it did on a Kaypro. You write, compile and run without leaving it, and what it produces is a CP/M program that runs under CP/M on the Z80 — it cannot see VICKY, the OPL2 or anything else on the K4510 side of the Tube, any more than a Kaypro could.

It is not shipped, because it is Borland's. Put TURBO.COM and TURBO.MSG into fs/CPM/H/3 — the folder that CP/M's drive P: is a shortcut to — and install it for a "VT100" or "ANSI" terminal, which is what JIM is. Then, from the K/OS prompt:

```
TURBO
```

TURBO is an alias that /SYSTEM/ETC/STARTUP.SAMPLE defines, for CPM K-TURBO: boot CP/M, go to P:, start Turbo, and — because the submit ends in EXIT — come all the way back to the K/OS prompt when you leave it. Chapter 9 explains that arrangement.



Turbo Pascal 3 starting on the Z80, its terminal drawn by JIM.

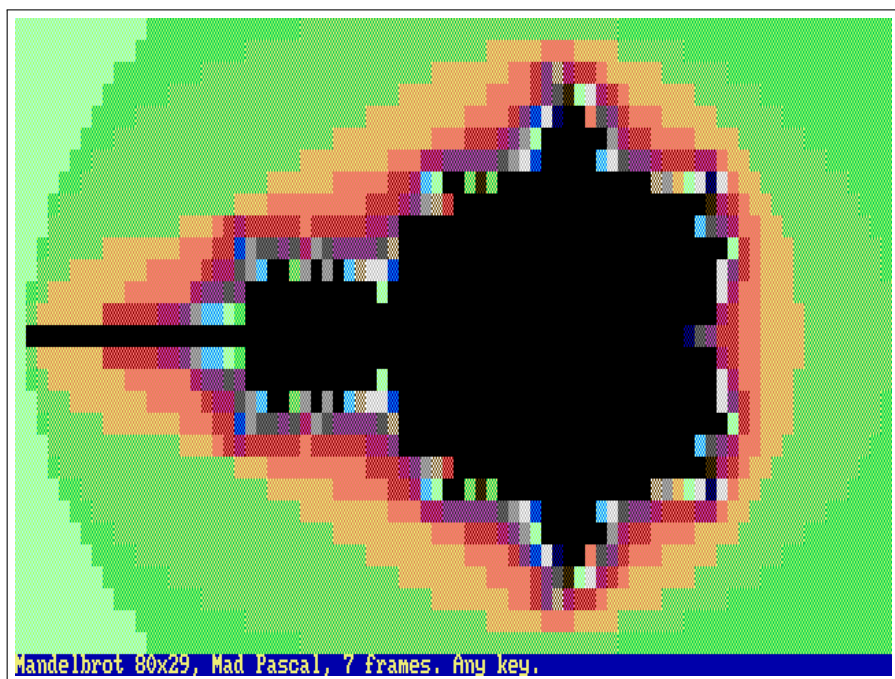
Inside, it is 1985. The editor is WordStar's: Ctrl-E, S, D and X move (the arrow keys are sent as those, so they work too; Home, End, PgUp and PgDn are not, and do not), and Ctrl-K then D leaves the editor for the menu. R runs the program in memory; O then C makes it compile to a .COM you can run from the CP/M prompt by name.

10.2 Mad Pascal, the cross-compiler

Mad Pascal is the other kind entirely — a cross-compiler, like the cc65 this machine's ROM is built with. You write on the desktop, mp compiles to 6502 assembly, MADS assembles it, and the result is a .prg beside

the source in /LANG/PASCAL that the shell's RUN loads like any other program — and, since 2026-09-08, one the machine can make for itself: PAS HELLO compiles HELLO.PAS in the directory you are standing in (Chapter 13). It is Tomasz Biela's compiler (MIT), a Turbo-Pascal-flavoured language with 8-, 16- and 32-bit integers, fixed and floating point, strings, records, pointers, units and inline assembly, made for the Atari and since taught the C64, the X16 and the Neo6502. The K4510 is its newest target.

PMANDEL



PMANDEL: the Mandelbrot set in text, sixteen colours through JIM, in 250 frames.

That is demo/pas/pmandel.pas: the Mandelbrot set, 78 by 28 cells in the machine's sixteen colours, in a little over four seconds at the desktop's 40.5 MHz — fixed point, 32-bit integers, forty lines of Pascal. PSIEVE is the classic sieve, ten passes timed by the frame counter;

HELLO prints, shows the colours, and echoes what you typed after its name. All of them are already compiled and sitting beside their sources in /LANG/PASCAL, so you can run them without installing anything.

Building

On the desktop, with FPC and a Mad-Pascal checkout (the installer patches its target table and rebuilds mp). It looks for the checkout in ~/Projects/neo6502_dev/Mad-Pascal unless MP_DIR says otherwise:

```
make pascal-install    # once; MP_DIR=... to point elsewhere
make pascal            # fs/LANG/PASCAL/*.PAS -> *.prg beside them
```

A new program is a file in fs/LANG/PASCAL; make finds it, and so does PAS on the machine itself. The compiled .prg files are committed, so a machine without the toolchain still runs them, and the test suite (test/pastest.sh) runs HELLO and PSIEVE through the ROM.

The target

Everything a Pascal program needs from the machine is in pascal/ of the repository, laid out as it lives inside a Mad-Pascal checkout: the runtime base (base/rtl6502_k4510.asm and base/k4510/), where @putchar writes to JIM, the terminal (Chapter 6); the SYSTEM and CRT units' machine halves (lib/*_k4510.inc); and a k4510 unit. The consequences for the programmer:

- Write and WriteLn go through JIM, so the CRT unit is the real thing: GotoXY, TextColor and TextBackground (the palette's constants: BLUE, YELLOW, LIGHT_GREEN...), ClrScr, ClrEol, InsLine/DelLine, WhereX/WhereY, CursorOn/CursorOff, ReadKey and KeyPressed on the keyboard device, Delay and Pause on the frame counter, and Sound on the machine's sound sequencer — which changed its contract when the OPL2 became the machine's chip: Sound takes a channel and a pitch in quarter-semitones and holds the note until NoSound. TextMode(0) puts the ROM's screen back.
- ParamCount and ParamStr come from the shell line (RUN HELLO one two, or just HELLO one two); ParamStr(0) is the whole tail.

- uses `k4510` gives every chip as a typed variable at its address — `VICKY[reg]`, `DMA_SRC`, `FS_CMD`, `SYS_FRAMES`, `MATH_F[]`, `TERM_CX` and the rest — plus `FarPeek/FarPoke` into the 256 MB through the 45GS02's flat addressing, `DmaCopy/DmaFill`, `Shell('DIR')`, `LoadFile/SaveFile` through the ROM, `WaitVBlank`, and `TermWrite` for raw escape sequences.
- Memory: code from `$0800`, the program's own; zero page `$22--$3F` for the compiler's registers and `$64--$A3` for its expression stack (the ROM keeps `$02--$21` and `$F0--$F9`); `$0300` the string buffer. Programs return to the shell with the ROM's state intact.

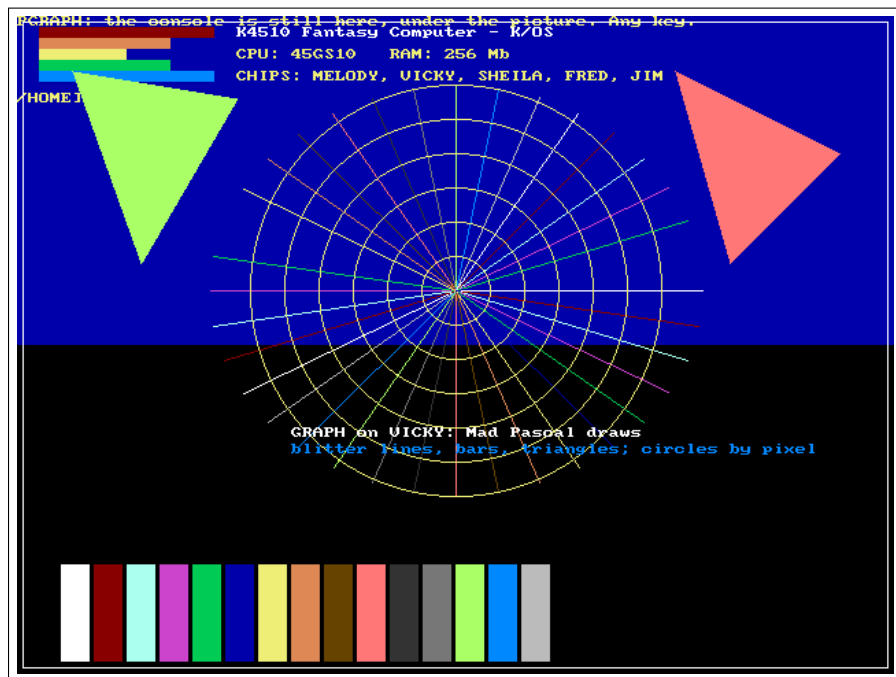
Floating point on the MATH unit

single (IEEE-754, 32-bit) does not run in software here. The runtime's add, subtract, multiply, divide, compare, Trunc, Round and Frac are the MATH unit at `$D700` (Chapter 15): each is a few register moves and one write to FOP, and the unit answers in the next cycle. PFLOAT times five thousand rounds of multiply, divide, add and subtract: 5 frames on the unit, 24 with the software library (assemble with `mads -d:SOFTFLOAT=1` to get it back for comparison). The transcendentals are in the `k4510` unit — `MathSqrt`, `MathSin`, `MathCos`, `MathTan`, `MathAtan`, `MathAtan2`, `MathExp`, `MathLn`, `MathPow`, `MathFloor` — one register write each. SYSTEM's own `Sqrt/Sin/Cos/ArcTan/Exp/Ln` on single run on the unit too: the installer patches `system.pas` under `{ifdef k4510}`.

Graphics on VICKY

uses graph draws on VICKY's layer 1: a `640×480` 8-bit bitmap at `$200000` — the one BBC BASIC's ULA uses too — over the console, which stays visible wherever the picture is transparent (colour 0), as in a BBC MODE. `InitGraph(0)` switches the video to 480 lines and clears the bitmap; `CloseGraph` puts the console's mode back. `PutPixel` and `GetPixel` go through the 45GS02's flat addressing (the MEGA65 multiplier does the row arithmetic); `Line`, `LineTo` and the K4510 additions `DrawLine`, `FillBar` and `FillTriangle` are the blitter's LINE, FILL and TRIANGLE operations — one register write each; `Circle`, `Ellipse`, `Rectangle` and

the rest of Mad Pascal's generic GRAPH build on those; OutTextXY writes with the console's font. PGRAPH is the demonstration.



PGRAPH: bars, a star and triangles by the blitter, circles by pixel, text from the ROM's font — and the console's own line still on top.

Chapter 11

The Editors

Two of them, because they answer different questions. EDIT is the one to reach for when a file wants a line changed and you want to be out again in ten seconds. VI is the one for a file too big to think about, and it is modal, so it expects you to have met vi before.

Both draw through JIM, the VT100 in hardware (Chapter 6), and both take their keys raw from the ROM — an arrow arrives as one byte rather than an escape sequence to unpick, with a bit beside it saying that it *is* an arrow and not the accented letter that shares its code. Neither needs the Tube.

11.1 EDIT

EDIT name opens a file, or starts an empty one if there is no such file yet. The whole text sits in memory below the program, so it holds about 22 KB — ample for a `STARTUP.BAT`, a `.SUB`, a BASIC listing or a letter. There are no modes: what you type goes in, and everything else is a control key.

Key	Does
arrows	move by a character or a line
Home, End	start and end of the line
PgUp, PgDn	a screenful
Enter	split the line at the cursor
Backspace	rub out to the left; at the start of a line, join it to the one above
Delete	rub out under the cursor
Ctrl-O or Ctrl-S	save
Ctrl-X	leave — twice on a changed file, to throw the changes away

Accented letters type as themselves. The bar along the bottom carries the name, a * while there are unsaved changes, where the cursor

is, and those last two keys, so there is nothing to remember. Long lines slide sideways rather than wrap.

11.2 VI

VI name is modal in the old way. In *normal* mode the keys are commands; a handful of them put you into *insert* mode, where what you type goes into the file; and Escape brings you back. The : line at the bottom is the third place, for the ex commands. The cursor says which of the three you are in: a block in normal mode, a bar while inserting, an underline on the : line. Accented letters go in like any other, in insert mode and after r.

Modes

Key	Enters insert...
i	before the cursor
a	after the cursor
I	at the start of the line
A	at the end of the line
o	on a new line below
O	on a new line above
s	replacing the character under the cursor
S	replacing the whole line
C	replacing the rest of the line
<i>cmotion</i>	replacing what the motion covers; cc the whole line
Esc	...and leaves it, back to normal mode
:	opens the command line (from normal mode)

Moving

A count goes in front of nearly anything: 5j is five lines down, 10G is line ten.

Key	Moves
h j k l	left, down, up, right — the arrows do the same
w	to the next word
b	back a word
e	to the end of the word
0	to the start of the line
^	to the first non-blank on the line
\$	to the end of the line
gg	to the first line
G	to the last line, or to line <i>n</i> with a count
PgUp, PgDn	a screenful
/text	to the next <i>text</i> — a plain substring, not a pattern
?text	to the previous one
n	the same search again; N the other way

Changing

Key	Does
x	delete the character under the cursor
X	delete the one to its left
d <i>motion</i>	delete what the motion covers: dw a word, d\$ to the end
dd	delete the line
D	delete to the end of the line
y <i>motion</i> , yy	copy (yank) the same, without deleting
p	put what was deleted or yanked, after the cursor
P	put it before
rc	replace one character with <i>c</i>
~	flip the case of one character
J	join the next line onto this one
u	undo — unlimited
Ctrl-R	redo — also unlimited

Operators take any motion — d2w deletes two words, y\$ yanks to the end of the line — or double themselves to work on whole lines. A charwise operator **clamps to the line**: dw at the end of a line stops there

instead of eating into the next one. That is deliberate, and so is the plain-substring search: `/foo` finds `foo`, and `.` `*` `[` mean themselves.

The command line

Command	Does
<code>:w</code>	save
<code>:w name</code>	save under another name
<code>:q</code>	leave — refuses on a changed file
<code>:q!</code>	leave anyway, changes lost
<code>:wq, :x</code>	save and leave
<code>:s/old/new/</code>	replace the first <i>old</i> on this line (/g: every one)
<code>:%s/old/new/g</code>	the same on every line of the file
<code>:map keys result</code>	define a key sequence, in normal mode
<code>:imap keys result</code>	the same, in insert mode

The command word is read in either case, so `:Q` and `:WQ` work with the caps lock on — which is how most people arrive in VI from a BASIC.

Making jk leave insert mode

The usual mapping is the one most people want first, and it is typed at the `:` line exactly like this, with the Escape spelt out in angle brackets:

```
:imap jk <Esc>
```

From then on, typing `j` then `k` in insert mode leaves it, and the two letters never reach the file. A lone `j` still does: the editor holds it for one second waiting for the `k` — vim's own `timeoutlen`, and long enough for a deliberate “j, k” — and hands it over when nothing comes. `<CR>` is the other spelt-out key; everything else in the result is literal.

To have it every time, put the same line, without the colon, in `/SYSTEM/ETC/VI.RC`: VI runs that file once the text is loaded, one `ex` command per line, a line starting with `"` being a comment. `/SYSTEM/ETC/VI.SAMPLE` is one to copy, the way `STARTUP.SAMPLE` is, and the file is yours — it is not in the repository. If a mapping you have defined seems not to fire, the usual reason is that there is no `VI.RC` on that machine, only the sample.

Where the file lives. Nothing of it is in the 64 KB. Every line is a 256-byte slot in far memory — a length and up to 255 characters — and only the line under the cursor is brought down, fetched when the cursor arrives and written back when it leaves. So the ceiling is far memory rather than the address space: 32000 lines, and LOAD and SAVE go straight to and from a flat copy out there without the file ever passing through the CPU's view. Undo is a journal in the same place, which is why it is unlimited: one level would have cost the same as all of them.

A 1.28 MB file of 20000 lines — twenty times the whole address space — opens, edits at the far end and saves back byte for byte.

11.3 Editing from inside a BASIC

There are two cases, and they look alike, so here is the rule.

To edit the program you are writing, type *EDIT or *VI with nothing after it. BASIC saves the program to a temporary file, runs the editor on it, and loads it back when you leave — so what you type in the editor is what you LIST afterwards. Variables do not survive the round trip, exactly as with LOAD. Section 4.5 has the details for EhBASIC; Microsoft BASIC does the same with *VI (Chapter 5), and LOGO with EDIT "name (Chapter 8).

To edit any other file, put SWAP in front:

```
*SWAP EDIT NOTES.TXT
```

A program loaded from the shell lands on top of whatever is in memory, so a plain *EDIT NOTES.TXT would run the editor over the program you were writing. SWAP (Chapter 2) puts the machine away first — program, variables, screen and all — runs the editor on a clean one, and gives yours back when the editor exits.

So: no file name, no SWAP needed, it is done for you; a file name, SWAP first.

Chapter 12

RX: the Machine's REXX

/STARTUP.BAT is a list of commands, and a list of commands is enough right up to the moment something has to be decided. RX is the answer to that: a REXX for this machine, in the Amiga's tradition rather than the mainframe's.

```
RX HELLO
```

runs HELLO.RX. So does typing HELLO on its own, because the shell's rule for an unknown word — try it as a program on disk — takes a script as readily as a .prg. The scripts that come with the machine are in /LANG/RX.

12.1 Why REXX, and what ARexx was

ARexx on the Amiga was two things at once. It was the REXX language — readable, typeless, and forgiving in the way a scripting language for people who are not programmers has to be. And it was the *port* system: every serious Amiga program published a port, so ten lines of REXX could drive a paint program and a terminal in the same script. The second half is why anybody remembers it.

Both halves are here. The language is demo/rexx.c, 37 KB loaded at \$0800 because it wants nearly the whole program area, and it is a real one: strings and whole numbers, PARSE with its templates, DO in all its forms, SELECT, internal functions with PROCEDURE EXPOSE and recursion, compound (stem) variables, SIGNAL, and about fifty built-in functions.

The thing that makes it glue is REXX's oldest idea: **a clause the interpreter does not recognise is not an error, it is a command**, sent to whatever the current environment is.

```
ADDRESS COMMAND  
'DIR APPS'  
IF RC = 0 THEN SAY 'that worked'
```

12.2 The three environments

ADDRESS	Sends the line to
COMMAND	K/OS's own shell — exactly as if you had typed it. The result comes back in RC
TUBE	the co-processor's shell, with its output brought back in RESULT
<i>name</i>	a program's port (below)

12.3 Ports: the file is the mailbox

The Amiga's ports were message ports between tasks that were all running at once. This machine runs one program at a time, so that arrangement could not be copied — and the substitute is the obvious one: a file.

A script writes its request to `/LANG/RX/MAIL.COMD`, SWAPs to the program with that file as its one argument, and reads the reply from `/LANG/RX/MAIL.RPL`: a result code on the first line and the text after it. A program joins the scheme by honouring one argument, which is about twenty lines of work.

CHESS is the worked example, and it plays a whole game this way: NEW, MOVE, GO, LEVEL, BOARD, FEN and STATUS. Its position lives in `/APPS/CHESS/PORT.GAM`, because a program swapped in starts fresh every time and the file is its memory. CHESS.RX in `/LANG/RX` plays against the engine and draws the board itself from what comes back.

12.4 What is in /LANG/RX

A language with nothing written in it proves nothing, so:

HELLO.RX	the smallest one
MACHINE.RX	the machine reading its own registers
HOSTINFO.RX	the host asked about itself, over the Tube
WC.RX	a word count
SUMFILE.RX	a column adder
GUESS.RX	a guessing game
WUMPUS.RX	Hunt the Wumpus, in 150 lines
CHESS.RX	a game of chess played through the port

Games are good tests. Wumpus alone found three real faults in the interpreter on the day it was written: EXPOSE of a stem was *assigning* it, which silently threw away every member of the caller's map; LEAVE stopped at the first END it met rather than the loop's, so leaving from inside a SELECT went nowhere; and a plain DO group was counting as a loop. None of them would have shown up in a script that only ran commands in order.

One thing to know about running programs from a script. A program started by the shell loads at \$6000 — through the middle of the interpreter. So RX asks the filesystem whether the first word of a command names a file: a built-in shell command runs directly and its output stays on the screen, and a program is swapped in and out around the interpreter. That is also why RX uses SWAP -k (Chapter 2): the ordinary SWAP would restore the screen and throw away what the program had just printed for the script to read.

Chapter 13

The Linux Underneath

Every emulated machine sits on a real one. On most of them that fact is carefully hidden; here it is a feature, because the real machine has compilers on it and the fantasy one is being used to write programs.

This chapter is about the seam — the one character that crosses it and the words that use it — and then about the Linux itself: what is on it, how it gets onto a stick or beside another system, and how it boots.

13.1 ! — the Linux prompt, from the machine's prompt

A line beginning with ! is not the machine's:

```
!ls -l
!git status
!nvim notes.txt
```

runs that command on the Linux the machine is standing on, with its output drawn on the machine's screen. A bare ! with nothing after it gives you an interactive shell there; leave it the way you leave any shell.

It is carried on the Tube (Chapter 6), which is the part of this machine already built to have another processor's console on the glass: the host's shell is started on the same pty the co-processors use, JIM renders it, and the ROM's own key loop feeds it. So the colours stay the machine's, full-screen programs work — nvim, htop and tmux run in the window — and ls --color lands in the machine's palette. The shell is told its terminal is **xterm-color**, and it speaks UTF-8: JIM draws what it can in the machine's character set and a near likeness of the rest, and an accented letter you type arrives as one.

SSH [user@]host is the same thing for a computer across the network: an ssh session, drawn by JIM, through the Linux's own ssh.

Locking the door. On a machine a child uses, the way into Linux may be a way into trouble. `linux = locked` in the menu file (Section 1.7) turns `!`, SSH and the menu's telnet row away with a message, and `consoles = locked` makes the Linux consoles unreachable from the keyboard. PAS and CC still compile.

What this means for the container. The sandboxed flavour (`linux/podman.sh`) is a wall on purpose: it is handed the display, the sound, the game controllers and one folder, which the machine sees as `/MNT/SHARE`, and nothing else. `!` inside it is real, and reaches the container — which is a small Debian with the compilers on it, and not your computer.

13.2 PAS and CC: compiling on the machine

The compilers live on the Linux side, so the machine reaches them the same way you would — but without you having to remember four options:

```
VI HELLO.PAS
PAS HELLO
HELLO
```

PAS `name` compiles `name.PAS` in the directory you are standing in with Mad Pascal, and leaves `name.prg` beside it. CC `name` does the same for `name.C` with `cc65`, using exactly the rule the machine's own Makefile uses for its programs. Intermediates go to a scratch directory and never appear on your disk; the compiler's own errors appear on the screen, and its exit status comes back as the result code, so a script can test it.

That is the loop the machine owns: *edit the source on the machine, compile it from the machine's prompt, run it on the machine*. The compiler itself still runs on the Linux beside it, which is why “self-hosted” has an “almost” in front of it — but the part that matters when you are writing a program is all on this side of the seam.

13.3 What is on the other side

On a desktop, the other side is your own computer and has whatever you have put on it. On the K4510's own Linux it is a Debian built for the purpose, and the list is short because everything on it is there for a reason:

cc65, 64tass, nasm	the machine's own toolchain — it can rebuild itself
FPC + Mad Pascal	what PAS runs
git, neovim	so that work done here is work
curl, ssh	what ftp://, sftp:// and SSH go through
tek40xx	a Tektronix 4010 terminal, below
telnetd	bound to the machine itself and nothing else
NetworkManager, Tailscale	the network: F7 → Host

The consoles. Ctrl+Alt+F1 is the machine; Ctrl+Alt+F2 to F6 are Linux text consoles, 80 by 25 in the IBM PC's own font, where the Tektronix lives; Ctrl+Alt+F1 comes back. (On a laptop whose F-keys are media keys, add Fn.) Quitting the emulator (Shift+Esc, or F7 → Quit) drops you to a Linux shell on the machine's own console rather than to nothing, and F7 has a *Shut down the computer* row because there is no desktop to go back to. The power button performs a clean shutdown. Closing the lid does what F7 → Host → *Lid closed* says: *keep running*, to begin with, or *suspend*. (Quit to the Linux shell and nothing is keeping the machine awake: there the lid suspends.) The consoles speak UTF-8 and follow the keyboard layout chosen in F7.

The network. F7 → Host shows the computer's name and address, and *Wi-Fi / network setup* opens NetworkManager's own screen on a spare console to join a network; the machine is back when you leave it. A network joined once is remembered.

Telnet, both directions. TELNET 127.0.0.1 23 from the machine's prompt — or F7 → Host → *Telnet into the host* — logs in to the Linux underneath through the front door rather than through !, which is useful when you want a session that survives what the machine is doing. The daemon listens on the machine itself only, so there is no network path to it at all. Outbound, the network is real: TELNET reaches a BBS, and URLs work as Chapter 2 describes.

13.4 On a stick

```
sudo ./linux/build-live.sh
```

makes the image, and `dd` puts it on a USB stick. At boot a menu offers the keyboard layout, and then the whole system is copied into RAM, so the stick is not being read while you work. A persistence partition on the same stick keeps what should survive a restart: the machine's home directory — `k4510.cfg`, the menu file, your saved states and the machine's disk with your files on it — the networks you have joined, and Tailscale's identity. Leave the stick in.

The laptop's own drive is not there. The stick's kernel command line blacklists the storage drivers absolutely, so the laptop's disk is not mounted, not visible, and not writable by anything on the stick. That is not a policy setting you could turn off by accident; the machine cannot see it.

13.5 Beside another system

A computer that already runs Linux can have the machine as a second choice at power-on. Make an empty partition of 2 GB or more on its internal disk and label it `K4510`; boot the computer's usual Linux with the stick plugged in; and from the stick run

```
sudo ./install-k4510.sh
```

It formats that partition the first time (and never again), copies the system onto it, arranges for your settings and files to be kept in the rest of the partition, and adds two entries to the computer's boot menu:

K4510 Fantasy Computer

the machine, quietly: the machine's logo while Linux starts and nothing else, then the prompt.

K4510 (text boot)

the same system, showing Linux's messages as it starts — the one to choose when something needs looking at.

It refuses a partition on a USB disk, and one on the stick itself, and it touches nothing of the other system but its boot menu. Run it again from a newer stick to update: the system is replaced and your settings and files are kept. The stick is only needed for the install.

The boot menu. The computer boots whichever of the two systems was used last, so the other system's own updates, which restart it, come back to it. With the menu set to stay hidden, as it is on the computer this book was checked on, the computer starts at once; Esc as it starts shows the menu. That is the other system's GRUB setting (GRUB_TIMEOUT_STYLE=hidden, GRUB_SAVEDFAULT=true in /etc/default/grub), and the installer leaves it as it finds it.

The whole system is read into RAM at boot here too — everything on the K4510 partition, which is why nothing large should be kept on it beside the system itself.

13.6 A Tektronix beside the machine

tek40xx (Ian Schofield's, GPL-3) is a Tektronix 4010/4014 storage-tube terminal that is also a telnet client. It is on the K4510's Linux and in the container because of what a storage tube is *for*: a PiDP-11 answers on telnet ports, and a machine of that vintage deserves a screen of that vintage to draw on.

```
tek HOST [PORT]
```

on the second console — full screen on a bare console, in a window under a desktop. And without any PDP-11 at all:

```
tekplay NAME|FILE...
```

plays Tektronix plot files at a 9600-baud pace, which is the right speed to watch one being drawn. With no argument it plays every plot that ships; HOME clears the screen and END quits. Nothing in the emulator changed for any of this: it is a second program on the same computer, which is exactly what a second terminal was in 1975.

13.7 From another computer

The machine's Linux answers ssh (as the user k4510), which is how the menu file is edited once the machine is locked, and how a screenshot is taken from across the room:

```
ssh k4510@machine k4510-shot
```

writes one, as PrtSc would, into shots/.

Part II

Programmer's Guide

Chapter 14

Memory

14.1 The 64 KB view and the 256 MB behind it

The 45GS10's program counter is 16 bits: code executes inside a 64 KB window. The 256 MB of physical memory is reached three ways:

- **Flat pointers:** LDA [ptr],Z and the Q forms read and write any byte of the 256 MB directly. Data never needs banking.
- **Bank registers** (\$D600, one per 8 KB block): write a 28-bit base and that block of the window shows that memory. Bytes 0–2 set the base; byte 3 switches (bit 7 = off).
- **DMA** (\$D200): copy, fill, swap, line, triangle — instant, physical addresses.

14.2 The CPU view, unmapped

\$0000-\$01FF	system	zero page and stack
\$0200-\$07FF	system	the ROM's data and C stack, and two bytes worth knowing (below)
\$0800-\$CFFF	user	50 KB for programs, ROM-free at launch
\$A000-\$BFFF	ROM	<i>the sideways window; RAM underneath</i>
\$C000-\$CFFF	ROM	<i>RAM underneath, bankable</i>
\$D000-\$DFFF	I/O	always I/O, whatever is banked
\$E000-\$FEFF	ROM	<i>RAM underneath, bankable</i>
\$FF00-\$FFFF	stub	always ROM: system-call stub and vectors

Two bytes of system state are fixed, so that programs can rely on them:

\$022E — a script is running

non-zero while EXEC (and so /STARTUP.BAT) is feeding the shell. A program that would wait for a key — TYPE's "more", say — should not, because there is nobody there to press it.

\$03FF — the result

the shell's result code: 0 for success. A program sets it to say it failed, which is what an RX script reads as RC.

14.3 Programs

A .prg begins with two addresses, where it loads and where it starts, and the shell honours both. The C programs of /SYSTEM/BIN load at \$6000, Mad Pascal's at \$0800, the two BASICs at \$7000, and MONITOR at \$E000, in the RAM under the ROM — so that the memory a monitor is there to look at, \$0800 to \$CFFF, is left exactly as it was.

14.4 System calls

The page \$FF00 is always the ROM, whatever is banked, and its jump table is the whole of the interface a program needs:

\$FF80	CHROUT	print the character in A
\$FF83	CHRIN	wait for a key; it comes back in A
\$FF86	GETIN	a key in A, or 0 if none is waiting
\$FF89	LOAD	name at (\$F0), to \$F2--\$F5; status in A, size in \$F6--\$F9
\$FF8C	SAVE	name at (\$F0), from \$F2--\$F5, length \$F6--\$F9
\$FF8F	SHELL	run the line A/X points at, as if it were typed
\$FF92	VIDEO	put the ROM's video mode and palette back after a program drew
\$FF95	ARGS	what followed the program's name: (\$F0), length in A

Anything handed to a system call must lie below \$A000: during the call the ROM is banked in over \$A000--\$FFFF, and a line kept up there would

read as the ROM's bytes. SHELL copies the line into the shell's own buffer before running it, so the program's copy is left alone.

14.5 RAM under the ROM

`rom_out()` (or three pokes to the bank registers) banks blocks 5–7 onto the RAM beneath the ROM: a program then owns \$0800–\$CFFF + \$E000–\$FEFF = 57 KB. Every system call still works, because the calls go through the \$FF00 stub page, which banks the ROM in and out around them.

14.6 Sideways ROM

The operating system is bigger than its half of the 64 KB — so, like the BBC Micro before it, the machine grew *sideways*. The ROM file is a 24 KB base image plus appended 8 KB banks paged into the \$A000–\$BFFF window: bank 0 is the base image itself, banks 1 and 2 hold the colder commands, and bank 3 has the line editor, its history and the directory commands. Chapter 3 says which bank every command lives in. The shell pages the right bank in around each call, and up to sixteen banks — 128 KB of operating system — fit in the scheme.

The ROM is full, and it is kept that way on purpose: a command whose work can be a program on the disk becomes one — TYPE and the monitor did in September 2026 — because a program costs nothing to load (the disk is the host's, and on the K4510's own Linux it is in RAM) and can be replaced without touching the ROM.

The calls go one way, and this is the rule to hold on to if you write code of your own for a bank: code in a bank may call resident code as freely as it likes, but resident code must never call into a bank. At that address a different bank holds something else entirely, and the dispatch is the only thing that knows which bank is in. A routine that everything uses belongs in the base image, whatever it costs there.

14.7 RAM under the I/O page

A MAP of block 6 hides \$D000-\$DFFF and exposes RAM: with the ROM also banked away a program owns a contiguous field from \$0800 to \$FEFF — 61.75 KB of the 64. The trick that makes it safe is that MAP is an *instruction*, not a register: the program that hid the I/O can always ask for it back, so there is no way to lock yourself out. (The far-call gate is unreachable while block 6 is mapped; unmap before far calls.)

14.8 Programs bigger than the window

The K4SG executable format loads segments to any physical address and the far-call gate (\$DF00) calls between them: a plain JSR into slot n banks descriptor n 's code in, and the callee's RTS banks it back out. SEGDEMO shows two overlays sharing one address.

Chapter 15

The I/O Page

One page, \$D000-\$DFFF, always visible. The authoritative register-level reference is the machine's own headers, and Appendix A is generated from them at build time — the way the Neo6502 handbook generates its keyword reference from the firmware source. This chapter is the map of what lives where:

\$D000	VICKY	video: layers, palette, blitter, sprites, SHEILA
\$D100	input	keyboard FIFO and its status, the held keys, the mouse, break-pending
\$D200	DMA	copy / fill / swap / line / triangle
\$D300	storage	the host filesystem: open, read, dir, chdir...
\$D400	OPL2	the YM3812: address, data, status — the AdLib's arrangement
\$D500	system	clock, RTC, frame counter, DUMP, the settings the ROM reads, the sound sequencer at \$D5E0
\$D600	banks	the eight bank registers, masks at \$D620
\$D700	MATH	IEEE floats, transcendentals, math lists, mul/div
\$D800	Tube	the co-processor: status, byte in, byte out, start/stop
\$D900	N:	the network: four channels, tcp://, http(s):// and tnfs:// (see core/net.h)
\$DA00	JIM	the terminal: a VT100/ANSI in hardware, drawing on the console's screen (see core/term.h)
\$DF00	far gate	call slots, descriptor table pointer, depth

15.1 A split screen: SHEILA

The Apple II put four lines of text under its graphics; C64 programmers did the same with a raster interrupt timed to the line. On this machine the CPU does nothing at all: SHEILA, VICKY's display-list coprocessor (the Amiga's copper is its ancestor), runs a short list from memory at

every frame, in step with the raster, and a register it writes takes effect on the line about to be drawn. A split is six instructions:

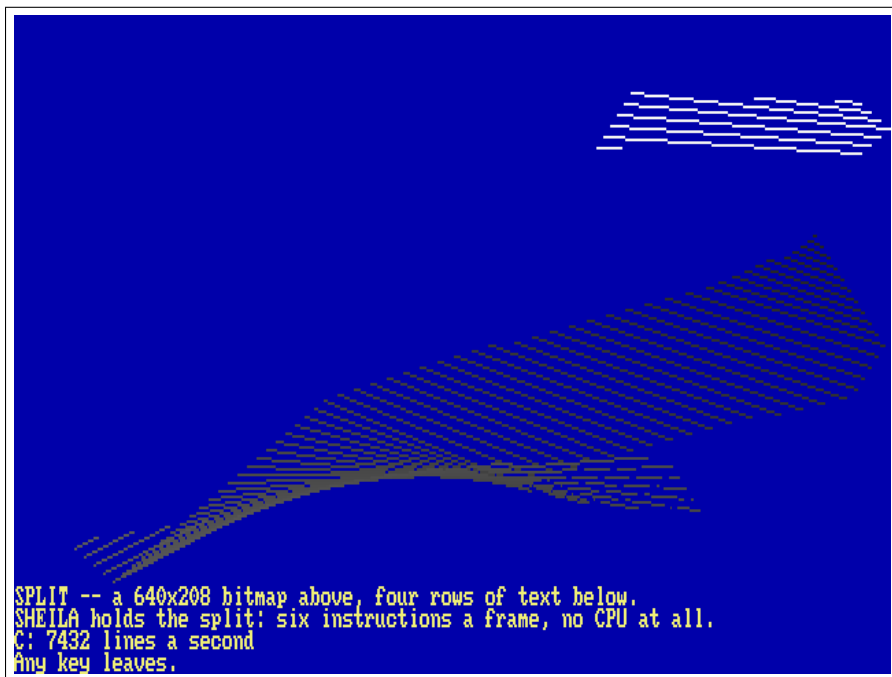
```
MOVE $10,$00    line 0: layer 0, the console's text, off
MOVE $20,$19    layer 1, a 640-wide 8 bpp bitmap, on
WAIT 416        26 text rows of 16 lines: the split
MOVE $20,$00    the bitmap off
MOVE $10,$07    the text on: rows 26-29 are text
END             and again from the top, next frame
```

Each instruction is four bytes — the operation (1 WAIT, 2 MOVE, 0 END), then its operands — and the list is anywhere in memory, a 28-bit address in \$D060–\$D063, started by 1 in \$D064. A MOVE is exactly a write by the CPU, so any VICKY register can change mid-frame: the mode, the scroll, a layer, the palette. Two rules: line numbers count the glass, 0–479, whatever the mode (in a 240-line mode each line is drawn twice, so split on an even one); and **nothing turns SHEILA off after a program**, so a program that turns it on turns it off when it leaves.

SPLIT is the demonstration, and it has two twins that draw the same ribbon of lines — endpoints bouncing, the line sixteen behind erased — as fast as they can, and say how fast in the text band: /LANG/EHBASIC/EX/SPLIT.BAS, with EhBASIC’s own LINE, and /LANG/MSBASIC/SPLIT.BAS, where Microsoft BASIC, which has no graphics words, sets the blitter’s registers with POKE. At the 40.5 MHz a host starts at:

Language	lines/s	how it draws one
C (SPLIT)	7400	fifteen register writes
EhBASIC	330	LINE, one blitter operation
Microsoft BASIC	140	ten POKEs in a subroutine

The blitter draws a line in no time at all, so the numbers are the languages: what a BASIC spends is its own arithmetic, its arrays and its line lookups, and three hundred lines a second is a good deal more than a game of 1985 asked for. The split itself costs every one of them nothing. A faster clock (F7, or SETUP) scales all three.



SPLIT: blitter lines above, the console's last four rows below, and SHEILA holding the line between them.

Chapter A

The Registers

Everything the machine's devices do, they do through the I/O page. This appendix is the register-level reference, and it is not written by hand: `doc/guide/mkregs.py` lifts it out of the machine's own headers at build time, so what you read here is what the emulator was compiled against. When a device gains a register, this appendix gains it in the next build — which is the only way a reference of this kind stays true.

Nothing in it is reworded: every word below is a word from a header. What the generator decides is only which column a word belongs in — the address on the left, the register's name after it, the comment's own sentence following. Where a comment's own spacing was doing the work of a table, that table is kept exactly as it was written, in monospace, because there it is the spacing that carries the meaning.

The voice is still the voice of working code: terse, occasionally opinionated, and using the machine's own shorthand — LE for little-endian, R: and W: for a register that reads and writes differently, \$ for hex. Addresses are given as an offset inside the device's block where the block's base is obvious, and in full where it is not.

Chapter 15 is the map of which device lives where; this is what is inside each one.

A.1 The I/O page

Generated from `core/io.h`.

The OPL2, wired the AdLib's way, so every AdLib register list means what it

The OPL2, wired the AdLib's way, so every AdLib register list means what it says here. The machine's one sound chip.

\$D480	<i>W ADDR</i> the register to write next R STATUS bit7 IRQ, bit6 timer 1 expired, bit5 timer 2 expired
\$D481	<i>W DATA</i> write it; R the last value written to that register
\$D482	<i>R ID</i>
\$02	= an OPL2 is fitted

The status bands and the clock's format, for the guest at \$D52D-\$D52F

The status bands and the clock's format, for the guest at \$D52D-\$D52F. \$D521 was the natural home and is FULL: all eight bits are spoken for (CP/M-by-name, margin, STARTUP.BAT, the bands, the mode request, and three of the video mode). So these are their own bytes, and there is room after them for whatever the menu grows next.

\$D800 status: bit 0 a co-processor is up, bit 2 the host shell is fitted, bit 3 a UCI chess

\$D800 status: bit 0 a co-processor is up, bit 2 the host shell is fitted, bit 3 a UCI chess engine is fitted (K4510_UCI, or stockfish where it usually lives), bit 7 a byte waits. \$D803 program: 1 BBC BASIC, 3 CP/M, 4 the host shell ('), 5 the chess engine, 2 stop. \$D804-7 the command string's address for program 4 (empty = an interactive shell), \$D808/9 its rows/columns.

\$D53A R BATTERY the host's battery, for the status band: charge in % in bits 0-6

\$D53A R BATTERY the host's battery, for the status band: charge in % in bits 0-6

(0-100), bit 7 set on AC power or charging, \$FF = no battery (a desktop, the browser build). The frontend reads it from the host every ten seconds; K/OS draws "BAT nn%" with an arrow beside the MHz.

Bank registers and the far gate

BANK registers: \$D600 + 4n, n = 0..7, one per 8 KB block of the CPU view.

bytes 0-2 physical base bits 0-23 (little-endian); they only set the base
 byte 3 bits 24-27 of the base, and bit 7 = OFF. Writing byte 3 switches
 the block: bit 7 clear = on, set = off. (So STQ works, and a
 byte-wise save/restore never switches a block on by accident.)
 Reads give the base; byte 3 reads \$FF while the block is off.

\$D620 read: bit n = block n banked
 \$D621 read: MAP mask (bit n = MAPped)

A banked block resolves phys = base + (cpu & \$1FFF). MAP rewrites all eight
 blocks, so the ROM's "MAP off" at program exit clears the banks too. The I/O
 page \$D000-\$DFFF and the stub page \$FF00-\$FFFF stay visible whatever is banked,
 so banking blocks 6 and 7 reveals the RAM under the ROM only. FAR gate: JSR
 \$DF00 + 4n calls descriptor n of the table at FARTAB.

\$DF00-\$DF7F 32 call slots
 \$DFF0 return gate (RTS lands here)
 \$DF80-\$DF83 **FARTAB** 28-bit pointer to the descriptor table (read/write)
 \$DF84 read: nesting depth
 \$DF85 read: last error (1 overflow, 2 underflow, 3 bad slot); write clears
 descriptor, 8 bytes: base[4] block flags entry[2]; flags bit0
 leave banked on return, bit1 do not bank (long jump to resi-
 dent code). A/X/Y/Z pass through both ways: cc65 __fastcall__
 works across it.

The MATH unit

MATH unit. Results are ready the cycle after the write that triggers them.

\$D700-\$D71F **F0..F7** eight IEEE-754 single registers, little-endian, read/write
 \$D720 **FOP** write = execute; low 5 bits op, see below
 \$D721 **FARG** (dst < 4) | src, register numbers 0..7, write before FOP
 \$D722 **FFLAGS** from the last op: bit0 result zero, bit1 negative, bit2
 NaN/inf
 \$D724-\$D727 **FI** int32 for ITOF / FTOI

ops: 0 MOV Fd=Fs 1 ADD 2 SUB 3 MUL 4 DIV (Fd = Fd op Fs)
 5 SQRT 6 SIN 7 COS 8 TAN 9 ATAN 10 EXP 11 LOG 14 ABS 15 NEG 16 FLOOR 17 ROUND (Fd =
 f(Fs))
 10 ATAN2 (Fd = atan2(Fd,Fs)) 13 POW (Fd = Fd^Fs) 21 FMOD (Fd = fmod(Fd,Fs))
 18 CMP flags from Fd - Fs, registers unchanged
 19 ITOF Fd = (float)FI 20 FTOI FI = (int32)Fs, truncated

	Math list – a program for the unit in RAM, run with one write:
\$D728-\$D72B	MLPTR 28-bit pointer to the list
\$D72C	MLRUN write = run from MLPTR until END or a STOP fires
\$D72D	MLSTAT 0 reached END, 1 a STOP fired, \$FF runaway (65536 ops)
\$D72E,\$D72F	MLCNT 16-bit counter for DJNZ, read/write
	list ops, 2 bytes each (op, arg) unless noted; ops 0..21 are the FOP ops with arg = (dst«4) src, and in addition:
\$80	END
\$81	STOPNEG stop if last flags negative
\$82	STOPPOS if not negative
\$83	STOPZERO
\$84	STOPNZ
\$85	JUMP arg (signed, in ops from the next op)
\$86	DJNZ arg MLCNT–, jump if nonzero
\$87	STOPFIGE arg stop if FI >= arg (unsigned byte compare on the low byte, FI clamped)
\$88	LDF arg=dst«4, then 4 bytes: IEEE single immediate into Fdst (6-byte op)
\$89	LDI , then 4 bytes: int32 immediate into FI (6-byte op)
\$8A	LDMS arg=dst«4, then 4 bytes: 28-bit address of a Microsoft-format float (exponent excess-128, mantissa1 with sign in bit 7, mantissa2, mantissa3 – EhBASIC's packed variable format); converted into Fdst (6-byte op)
	MEGA65-compatible integer unit (same addresses as the MEGA65):
\$D770-\$D773	MULTINA
\$D774-\$D777	MULTINB (unsigned 32-bit, LE)
\$D778-\$D77F	MULTOUT = A * B, 64-bit
\$D76C-\$D76F	DIVOUT integer part of A / B
\$D768-\$D76B	fractional part (32.32)
	recomputed on every write to an input byte; B = 0 gives all-ones.

System registers, and the SIDs

SYS registers (read-only unless noted):

\$00,\$01 CPU clock, kHz, LE (40500)

\$02,03	physical RAM, MB, LE (256)
\$04	read: latch the host clock into \$05-\$0C and return 0
\$05	sec
\$06	min
\$07	hour
\$08	day
\$09	month
\$0A,0B	year LE
\$0C	weekday (0=Sun)
\$0D,0E,0F	frames since reset, 24-bit LE (vblank count)
\$10-\$1F	version string, NUL-terminated
\$20	ROM base page (e.g. \$A0 for a 24 KB ROM)
\$F0	write: DUMP – the host writes dumps/dump-NNN.txt (machine state, screen, PC history, keys, the shell log); read: the number of the last dump
\$F1	write: append a byte to the shell log (the ROM logs command lines and DUMP notes)
\$F2	write 1/0: automatic dump every 900 frames (15 s) on/off (on at reset on the desktop, off on the Pi; DUMP or DUMP ON also arms the per-instruction PC recorder); read: the setting
\$23	read: the CPU clock setting in force (0 = 40.5 MHz, 1 = 30, 2 = 20, 3 = 15, 4 = 10); write: ask for one – the host applies it next frame (BENCH sweeps them)
\$24,25	audio gaps LE: callbacks that found the ring empty since last cleared; any write clears

The keyboard

Keyboard: a FIFO of key-down events. Printable keys arrive as ASCII (\$20-\$7E, already shifted/dead-keyed by the host layout on the desktop); control keys as ASCII controls; everything else as \$80+ codes.

bit7 event available; bit0 shift, bit1 ctrl, bit2 alt held;

bit7 event available; bit0 shift, bit1 ctrl, bit2 alt held; bit6 the byte LAST READ from KBD was a key code (KEY_*), not a character; bit5 the byte WAITING is

a key code. The two share \$80-\$9F – KEY_LEFT and é are both \$82 – and these bits are how a line editor tells them apart.

— WATCH (\$D530-\$D535): who is trampling this byte? —————

\$30-\$33 28-bit physical address to watch

\$34 write 1 to arm; write 0 to disarm; reads back the armed state

\$35 hits since last arm (saturates at 255)

A CPU write to the watched byte – normal, MAPPED, banked or a flat 32-bit store – fires a DUMP (dumps/dump-NNN.txt, tagged “watch”) and disarms, so one bug produces one dump. DMA and the host itself do not trip it: the question WATCH answers is “which instruction wrote this”, and the dump’s PC history is the answer.

The Tube

The Tube (\$D800): Acorn’s answer, refitted. The HOST runs Richard Russell’s BBC BASIC interpreter (the vendored BBCTTY console edition, tube/bbcbasic) on a pty; the machine talks to it byte-wise:

\$D800 *R* status bit0 alive, bit7 a byte waits in \$D801

\$D801 *R* next byte from the co-processor (pops)

\$D802 *W* a byte to the co-processor (its keyboard)

\$D803 *W* 1 start (spawn), 2 stop (kill)

The co-processor has its own flat 256 MB; PAGE/HIMEM live there, far beyond the 64 KB view. On the Pi the co-processor is the same interpreter (or RunCPM’s Z80, program 3) running on core 3 (core/tube_cp.c); an unfitted program leaves status reading 0. The console it talks to is JIM, the terminal at \$DA00 (core/term.h).

A.2 VICKY, SHEILA and the sprites

Generated from core/vicky.h.

VICKY

VICKY – the K4510 video chip.

Register block at IO_VICKY (\$D000), 256 bytes, byte-addressed. Every pointer is a 28-bit physical address into main RAM; there is no video memory. Rendering is per scanline into an 8-bit indexed framebuffer the frontend supplies; the frontend applies `vicky_palette_rgb()`.

\$00 **CTRL** bit0 display enable; the rest pick the mode. The glass is always 640x480 and the raster lines are always 0..479 – a smaller mode is drawn into it, doubled, and centred. bit1 columns halved (320), bit2 lines halved (240), bit3 a 200-line field (40 blank lines top and bottom), bit4 columns quartered (160; with bit1).

1	640x480	1 4	640x240
1 2	320x240	1 2 8	320x200
1 2 8 16	160x200	1 4 8	640x200

\$01 **BGCOL** background palette index (where nothing is drawn)

\$02 **RASTER** read: current line (low 8); write: raster-compare low

\$03 read: line high bits; write: compare high

\$04 **IRQSTAT** bit0 vblank, bit1 raster==compare, bit2 SHEILA IRQ op, bit3 sprite collision. Write 1s to acknowledge.

\$05 **IRQMASK** same bits; IRQ line = IRQSTAT & IRQMASK

\$06 **PALIDX** palette index for the write port

\$07 **PALR**

\$08 **PALG**

\$09 **PALB** – writing B commits the entry and increments PALIDX

\$0A-\$0D **SPRTAB** 28-bit pointer to the sprite attribute table (128 x 16 B)

\$0E **SPRCTL** bit0 sprites enable

\$0F reserved

\$60-\$63 **SHEILA** 28-bit pointer to SHEILA's list

\$64 **SHEILACTL** bit0 enable; the list restarts every frame at line 0

\$70-\$73 **BLTSRC** 28-bit

\$74-\$77 **BLTDST** 28-bit

\$78,79 **BLTW** width px

\$7A,7B **BLTH** height px

\$7C,7D	BLTSS source stride (bytes)
\$7E,7F	BLTDS dest stride
\$80	BLTOP 0 copy, 1 keyed copy (src 0 skipped), 2 fill (value = BLTSRC byte 0), 3 AND, 4 OR, 5 XOR, 6 LINE: from (LX0,LY0) to (LX1,LY1), colour = BLTSRC byte 0, into the BLTDST surface of stride BLTDS, clipped to 0..BLTW-1 x 0..BLTH-1 7 TRIANGLE: filled (LX0,LY0)-(LX1,LY1)-(LX2,LY2), same colour, surface and clip as LINE
\$81	BLTFLG bit0 H-flip, bit1 V-flip
\$82	BLTCMD write anything: go. Instant. Reads 0.
\$84,85	LX0
\$86,87	LY0
\$88,89	LX1
\$8A,8B	LY1
\$8C,8D	LX2
\$8E,8F	LY2 (signed 16-bit) Blits are 8 bpp (one byte per pixel) in this version.
\$90-\$9F	COLSS read: sprite-sprite collision bits, one bit per sprite (sprite n hit another sprite this frame). All 16 cleared on read of \$90.
\$A0-\$AF	COLSL read: sprite-layer collision bits (sprite n over a non-transparent layer pixel). All 16 cleared on read of \$A0. Sprite attribute entry, 16 bytes, in main RAM:
+0,1	X (signed 16)
+2,3	Y (signed 16)
+4..7	DATA 28-bit pointer
+8	CTRL bit0 enable, bit1 8 bpp (else 4), bit2 H-flip, bit3 V-flip, bits4-5 Z: drawn after layer Z (0..3)
+9	SIZE bits0-1 width 8/16/32/64, bits2-3 height 8/16/32/64
+10	PALOFFS (4 bpp: index = PALOFFS«4 pixel)
+11..15	reserved Pixel 0 is transparent. No per-line limit. 128 sprites.

SHEILA – the display-list coprocessor (Doc named it, 2026-08-22; the Amiga's copper is the ancestor). 4-byte instructions in main RAM, executed at the start of each scanline until a WAIT blocks. Register writes take effect for the line about to be drawn.

00	END stop until next frame
01	WAIT lo hi wait for line $\geq (hi \ll 8 lo)$
02	MOVE reg val write val to VICKY register reg
03	SKIP lo hi skip next instruction if line $\geq$ value
04	JUMP a0 a1 a2 continue at 24-bit address
05	IRQ set IRQSTAT bit2
	At most 256 instructions per line are executed (runaway guard).
	Layers 0..3 at $\$10 + n * \10 , 16 bytes each:
+0	LCTRL bit0 enable, bits1-2 mode (0 bitmap, 1 tile, 2 text8, 3 text32), bits3-4 bpp (0=1, 1=2, 2=4, 3=8), bits5-6 cell size (tile: 8/16/32/64 px square; text: 0 = 8x8, 1 = 8x16)
+1	LPALOFS palette offset for <8 bpp: index = (value $\ll$ depth) pixel
+2,3	SCROLLX 16-bit, pixels
+4,5	SCROLLY 16-bit, pixels
+6,7	STRIDE bitmap: bytes per row. tile/text: map entries per row.
+8..+B	DATA 28-bit pointer: pixels (bitmap) or glyph/tile set
+C..+F	MAP 28-bit pointer: the map
Map formats:	
tile	2 bytes/cell: bits 0-9 tile index, 10 H-flip, 11 V-flip, 12-15 palette offset (used for <8 bpp). Tile pixel data at DATA + index * (size*size*bpp/8), rows MSB-first packed.
text8	1 byte/cell: glyph index. 1 bpp 8xH glyphs at DATA + g*H. Colours from LPALOFS: index = (LPALOFS $\ll$ 1) pixel.
text32	4 bytes/cell: glyph lo, glyph hi (16-bit index), fg, bg – byte-wide palette indices per cell. bit7 of glyph hi = reverse.
Layer 0 is bottom. Pixel index 0 is transparent in every layer; BGCOL is the ground (text32 bg is never transparent). Changed 2026-08-22 from “opaque in the lowest layer” so SHEILA backgrounds show under text.	

A.3 The network

Generated from core/net.h.

The N: device

The network, three ways – each borrowed from the machine that did it first.

The Meatloaf rule (the C64's Meatloaf cartridge): a URL is a file name. A name beginning `http://`, `https://` or `tnfs://` that reaches the filesystem device (\$D300) for reading – LOAD, TYPE, CP, RUN, EhBASIC's LOAD, BBC BASIC's LOAD on the Tube – is fetched and served like a file. No ROM change: the ROM passes names through untouched and the host fetches.

TNFS (the Spectranet's file protocol, the one FujiNet and Meatloaf servers speak; UDP, port 16384): a `tnfs://host[:port]/path` is also a DIRECTORY. `CD tnfs://host/path` puts the machine's current directory on the server: `DIR` lists it, a bare name loads from it (so the REXX rule runs programs off the internet), `CD ..` climbs it, `CD -` comes home. The server is read-only from here.

The N: device (FujiNet's network device, as the Atari and Apple saw it), at \$D900, for programs that want a live connection: four channels, each a URL opened for reading and writing.

\$D900	<i>W</i> command
\$D901	<i>R</i> status (0 ok, 1 not found / refused, 2 i/o error, 3 bad command, 4 closed by the peer, 5 name too long, 6 not fitted)
\$D902	<i>RW</i> channel 0-3
\$D904-\$D907	NAMEPTR 28-bit -> URL, NUL-terminated
\$D908-\$D90B	ADDR 28-bit
\$D90C-\$D90F	LEN bytes requested; updated to bytes done
\$D910-\$D913	SIZE after OPEN: the content length, \$FFFFFF if unknown; after STATUS: bytes waiting
	commands: 1 OPEN (<code>tcp://host:port</code> a connection; <code>http://</code> , <code>https://</code> , <code>tnfs://</code> a file, fetched whole, then READ)
	2 READ (up to LEN bytes to ADDR, what has arrived so far; LEN = done, 0 = nothing yet)
	3 WRITE (LEN bytes from ADDR; tcp only)
	4 CLOSE
	5 STATUS (SIZE = bytes waiting; status 4 once the peer has closed and the bytes are gone)
	6 GET (the Meatloaf rule for programs: fetch the whole URL into ADDR, at most LEN; LEN = done, SIZE = total)

Reads never block the machine: a program polls, as it would a UART. Underneath: `core/net_plat.h` – sockets and curl on the desktop, Circle's stack on the Pi (no TLS there: https answers 6).

A.4 JIM, the terminal

Generated from core/term.h.

JIM

JIM, the terminal (\$DA00) – the Beeb’s third page, given a job: a VT100 with the ANSI colour and VT220 editing additions, in hardware – the way a real 8-bit machine got a serious terminal: a card, not a program. It draws on the VICKY text32 screen the ROM console uses, inside the geometry the ROM gives it, so the console and the terminal share one screen and one cursor. Anything that needs a terminal writes its byte stream here: the ROM for the Tube (CP/M programs set up for VT100/ANSI, BBC BASIC’s console edition) and TELNET for the BBSes (ANSI-BBS: CP437 glyphs, 16 colours).

\$DA00 *W DATA* a byte of the stream

\$DA01 *R STATUS* bit7 a reply byte waits; bit0 the stream moved the cursor since CX/CY were written

\$DA02 *R REPLY* the next reply byte (pops): answers to ESC[6n / ESC[c, and translated keys

\$DA03 *W KEY* a K4510 key code (io.h): its terminal bytes go to REPLY (arrows ESC[A.. or ESC OA.. in application mode, Home/End, PgUp/PgDn/Ins ESC[n~, Del \$7F, F1-F4 ESC OP.., F5-F12 ESC[15~.., everything else through unchanged)

\$DA04 *W CTRL* 1 reset (modes, attributes, cursor home; the screen kept)
2 clear the screen and home

\$DA05-\$DA0D *RW COLS ROWS OX OY CX CY FG BG STRIDE* the window: origin (OX,OY) cells, STRIDE cells per row

\$DA0E *RW FLAGS* bit0 cursor shown (blinking) bit1 read: application cursor keys (DECCKM) bit2 PETSCII mode bit3 THE STATUS BANDS BELONG TO THE PROGRAM. While it is set, K/OS lays the console around \$DA0F/\$DA16 instead of the user’s F7 heights, and stops drawing into the bands at all – no clock, no MHz, and cls() leaves those rows alone. The program draws them itself and MUST clear the bit before it exits, the way PETSCII mode must be cleared: leave it set and the shell comes back to a screen whose furniture nobody is maintaining.

\$DA0F *RW BANDTOP* rows in the top band while bit3 is set

\$DA16 *RW BANDBOT* rows in the bottom band while bit3 is set. Write the two, set bit3, then call VIDEO (\$FF92): K/OS re-lays the console

around them. Clear bit3 and call VIDEO again to hand them back. (JIM stores these and never reads them; the ROM does. They live here because this is where the console's geometry lives, and because the frontend rewrites the user's heights every frame, so a guest request has nowhere else to survive.)

`$DA10-$DA13` *RW* **BASE** 28-bit address of the text32 map (reset: `$030000`)

`$DA14,$DA15` *RW* **DEFFG** **DEFBG** the colours SGR 0 / 39 / 49 return to

Sequences: the VT100 set (cursor, ED/EL, DECSTBM, DECSC/DECRC, IND/RI/NEL, tabs, DECAWM/DECOM/DECCKM, DEC line drawing via ESC(0 and SO/SI, DSR, DA, DECALN, RIS), ANSI SGR 0/1/4/5/7/22/24/27/30-37/39/40-47/49/90-97/100-107 and 38;5;n / 48;5;n for n < 16, VT220 ICH/DCH/IL/DL/ECH/SU/SD/CHA/VPA, IRM, ESC[?25 cursor, ESC[s/u, DECSTR, DECSCUSR (ESC [n SP q: 0-2 block, 3-4 underline, 5-6 bar – the shape VI changes with its mode). Bytes `$80-$FF` are glyphs (CP437). UTF-8: ESC % G on, ESC % @ off (CTRL 1 leaves it). On, a UTF-8 sequence draws as its CP437 glyph (or a near one, or '?'), and a byte that continues no sequence is CP437 as before. The 'j shell and TELNET turn it on for their sessions.

A.5 Memory and the far view

Generated from `core/mem.h`.

The ROM window and the stub page

The ROM image lives in the top 64 KB of physical memory and is seen in the unmapped CPU view from `mem_rom_base` up; the physical RAM at `$A000-$FFFF` is "RAM under the ROM", revealed by banking blocks 5/7 onto `$A000/$E000` (K-05). The page `$FF00-$FFFF` always reads the ROM, whatever is banked: the system-call stub and the vectors live there.

Chapter B

Filing an Issue

This machine has one author, and an issue you file will, in all likelihood, be handed straight to an AI coding session to work on, as written. That is worth knowing, because it changes what a good report is: not a description of the fault, but the machine's own account of it. The machine can write most of that account itself, and the whole of this appendix is how to let it.

B.1 Before you try again

None of these is politeness. Each one removes a day.

Turn DUMP ON before you reproduce it. Then every fifteen seconds, and again the instant you type DUMP once it has gone wrong, the machine writes its whole state — registers, the screen as you see it, the last 4096 instructions, your keystrokes, and the log of every command you have typed — to `dumps/dump-NNN.txt` on the host. A dump taken while it is wrong is worth more than any description either of us could write, and DUMP ON means you cannot miss the moment.

Try it once with `k4510 --no-startup.bat`. A `/STARTUP.BAT` that has gone wrong looks exactly like a machine that has gone wrong, and telling them apart has cost this project a day before. Ten seconds of yours settles it.

Check this book. If the machine disagrees with a page here, that is still a fault worth filing — it is this book's, and it is fixed the same way. Say which page.

B.2 BUG writes the report

Then let the machine do the writing:

```
/] BUG
```

BUG interviews you — seven questions, one at a time, each answered in a line or two — and writes the finished report to

```
/SYSTEM/LOG/BUGREPORTS/BUG-YYYYMMDD-HHMMSS.TXT
```

making the directory if it is not there. Five lines it fills in itself, and they are the five most often got wrong by hand: which machine (the K4510 on its own Linux, or in a window on somebody's desktop), the exact build (INFO -v says the same — a trailing + means the emulator was built from an edited tree, which is exactly the report that will not reproduce), the screen you are in, the last dump, and when.

The questions are the ones a coding session would otherwise have to ask you: where you were (the shell, a BASIC, CP/M...), what you typed *exactly*, what happened, what you expected instead, and *why* you expected it — the line most reports leave out. The K4510 is a fantasy machine (page 109): there is no silicon it can be measured against, so “wrong” only means something next to what you were expecting and where that came from. It is also what separates a bug from a design decision from an error in this book, which are three different repairs.

BUG is a program rather than a command, so it reaches you everywhere the * escape does: *BUG from either BASIC, from the monitor, from CP/M.

DUMP first, then BUG. If the report's Dump line says there is none, the more valuable half is missing. Type DUMP while it is still wrong and run BUG again — the two travel together, and this is the one thing that is easy to do in the wrong order.

B.3 Where it goes

<https://github.com/mlongval/k4510/issues>

From the host, the report is in fs/SYSTEM/LOG/BUGREPORTS/ and the dump in dumps/: open an issue, attach both, and you are done — the report is the whole text of the issue. The form there asks BUG's questions again for anyone who has no report file: a fault in this book, a machine that

will not boot, a telephone. If you have no account there, the same two files in an email are worth exactly as much.

Feature requests and disagreements are welcome in the same shape — “what I expected” does the work whether or not anything is broken.

One thing to check before you attach a dump. It carries the shell log, which is every command line you typed in that session, and file names from your own disk along with them. It is a plain text file: open it and have a look before it goes anywhere public.

Chapter C

The Sound, and What It Took

Every other chapter of this book describes the machine as it is. This one describes how it got that way, because the sound took longer to get right than anything else in it and the route was instructive. Nothing here is needed to use the machine. It is here because a fantasy computer is a series of decisions, and the decisions about sound were the ones that most often turned out to be about something else.

It has one thesis, and it is worth stating before the evidence rather than after: **the sound was hard because keeping a real-time audio ring fed from a machine that runs in bursts is hard, not because emulating a SID is expensive.** Every fault the sound reported turned out to be somewhere else, and the chip emulation — the part everyone suspects — has never once been on the critical path.

If you are reading it to find out what happened to the SID chips: they are gone, and the last two sections say why. The reason is not the one you would guess from the six weeks described here, which is most of what makes it worth writing down.

C.1 Four chips, because it is a fantasy

The C64 had one SID. The plan for this machine was four, at \$D400 and every \$20 after it, for no better reason than that a machine which never existed may have whatever its author thinks it should have and four is a chord. Dag Lem's reSID — the emulation that has been the reference for thirty years — was vendored unchanged from VICE 3.3, wrapped in `core/sid.cc`, and mixed to 16-bit mono. The frontend fed a ring buffer a scanline at a time and an audio callback drained it at 48 kHz.

That took an afternoon and it worked. Everything after this section is the six weeks of it not quite working.

C.2 A SID is not a CPU

The first mistake was in the first version and is the kind only a fantasy machine can make: the SIDs were clocked at the CPU's clock.

On a C64 the two are the same crystal, so the question never comes up.

Here the CPU runs at 40.5 MHz and the SID is a 1 MHz part, and clocking reSID at 40.5 million cycles a second cost **21 milliseconds a frame** — more than the whole frame — and put every pitch out by a factor of forty-one. The fix was one line and the lesson outlived it: *the SID's clock is not the CPU's*, and every later change that touched the clock had to be told so again. The most recent time was in August 2026, in a benchmark that swept the CPU clock and forgot to tell reSID, and rendered every note at the wrong rate.

C.3 “Choppy choppy choppy” — which was not the sound

The machine's first outing on real hardware, a Raspberry Pi 3B+ on a television, produced the report that named the problem for the rest of the project: *the SID sounds on the RPi3 are very very bad, choppy choppy choppy*.

It was not an audio bug. The Pi was running the machine at about 17 ms against a 16.7 ms frame, and two recent changes had spent that slack — a debug recorder storing every instruction fetch, and a rule that made the I/O page win on every memory access, which put an extra comparison on every read and write in the machine. Neither was visible on a desktop. Both were fatal on the Pi, and what they broke was the sound, because the sound is the first thing that breaks when a frame runs long.

That is the pattern the rest of this appendix is about. **The sound is the machine's alarm bell.** Almost every time it rang, the fault was somewhere else.

C.4 Silence is not free

With a clean power supply and honest measurements, the Pi at 20 MHz sat 1.6 ms over its frame budget. Of the 18.3 ms, **3.4 was the SIDs** — all four of them, three of them silent.

reSID costs the same for silence as for sound: a chip that has been written to is a chip that must be clocked, whether or not anything is audible. So a chip is now clocked and mixed only once the machine has written to it since reset, which at the prompt is three chips' worth of frame given back. Note what that 3.4 ms was — four chips *clocked*, not four chips playing — and that it is the largest figure the sound has ever cost anywhere in this project.

That fix has a tail which is still in the machine: the cost is latched by the *first write*, not by whether anything is sounding. A program that pokes a SID register during setup and then goes quiet pays the full price for the rest of the session. On a desktop that is half a millisecond of sixteen and nobody notices. On a Pi with no slack it is worth knowing.

C.5 The ring that started empty

On the Pi the audio callback is served by the same core that runs the machine, once a frame, a block at a time. A frame makes 800 samples; the block was 1024; the ring started empty. So the ring's level lived between nothing and one frame, and a callback regularly found 800 real samples and 224 of silence — a gap, on a steady beat, at a frame rate the machine was comfortably holding.

Smaller blocks and a lead of three of them — 32 ms, which a music player does not feel — and the level never touches the floor.

It made things worse. At 15 MHz the gap count went from 40–48 in two seconds to 52–62.

C.6 Chips that play on without the machine

The lead failed because the model behind it was wrong, and the numbers said so plainly. At 15 MHz the Pi ran at 58–60 frames a second, not a locked 60. Audio was made only by the machine's frames, 800 samples

each. At 58 frames that is 46,400 samples a second against the 48,000 the device consumes. *Any* shortfall drains the ring and then gaps for ever after; a lead only postpones the first one. Even rounding the cycles per scanline down from 520.83 to 520 is a 0.16 % deficit on its own.

So the machine stopped tying its sound to its frames. Real hardware does not stop its sound chips because the processor stalled — a SID keeps oscillating whatever the 6502 is doing — and now nor does this one: after a frame, if the ring is below its target, the SIDs are clocked on *without the CPU* until it is full. The SID clock does not move, so pitch does not move; a slow frame sustains a note a fraction longer rather than cutting it.

This is the most important decision in the whole story, and it is the one that came back.

C.7 The meter that read zero

With the chips decoupled, and with the Pi's callback served every 128 lines rather than once a frame, the gap counter read **zero**, on three boots, at the default clock. By every instrument the project had, the sound was clean.

Doc: *both say 0 drops but I still can hear them.*

He was right and the meter was wrong. The gap counter counts callbacks that found the ring *empty* — it counts silence. And the decoupling exists precisely to ensure the ring is never empty. So across the entire band where a host is merely losing rather than drowning, the counter reads zero and there is nothing wrong with the sound except that a growing fraction of it was not made by the program. The SIDs were playing on without the machine. That is what he was hearing.

It is now counted. Beside the gaps there is a second number: how many samples the machine did not make and the chips supplied. On one host, at rows both instruments had called clean:

Clock	Frames	Gaps	Filled in
30 MHz	53 fps	0	36,464
20 MHz	60 fps	0	16,818
10 MHz	60 fps	0	1,609

of 96,000 samples in the two seconds measured. At 30 MHz *more than a third of the sound was not the program's*, and every meter said clean.

Clean now means three things at once, because any two of them can be true while the sound is wrong: sixty frames a second, no gaps, and under two per cent filled in.

The moral, three times over. A performance regression that presented as an audio bug. An audio fix that made the gap count worse. A gap counter that read zero because an earlier fix had guaranteed it would. Each time, the instrument was measuring something adjacent to the fault — and each time, the person listening to the machine was right before the numbers were.

C.8 A machine keeping somebody else's time

The newest finding, and the one that reframes the rest: on the desktop the machine was not keeping its own time at all.

The Pi has always paced itself against the wall clock, because its graphics layer blocks until the display flips and a frame that overran by a millisecond would wait for the next one and halve the frame rate. The desktop did the opposite — it left the pacing to the display's vertical sync and did none of its own, so the machine ran at whatever the compositor handed it. On one host, a 60 Hz display, that was **51.8 frames a second**, with the act of putting the picture on the glass taking 16.9 ms of a 19.3 ms frame.

A machine at 51.8 frames a second makes 51.8 frames of sound where the device wants 60. The chips filled in the missing seventh, exactly as designed, and it was audible — and the new counter measured it at 17 %. The machine now keeps its own 60 Hz on the desktop as it always has on the Pi, and presents when it is ready:

	Whole frame	Onto the glass
before	19.294 ms (51.8 fps)	16.867 ms
after	16.668 ms (60.0 fps)	0.260 ms

C.9 Where it stood, when it was decided

By the end of it the machine had four reSID chips in it and ran as many of them as you asked for, and the question of whether to keep them was open. It had been raised as a performance question, after a Raspberry Pi that would hold only 15 MHz. On that ground it was settled, and settled *against* replacement. **reSID was not the cost.** One chip sounding measured **0.039 ms of a 16.67 ms frame** on a desktop — a quarter of one per cent — and four chips written since reset came to about half a millisecond. The largest the sound ever cost was the 3.4 ms of §C.4, on the Pi, before silent chips stopped being clocked, and that was a book-keeping fault rather than a fidelity one. Nothing measurable was to be bought by removing them.

Where the question was real was identity. The SID is a part this machine inherited rather than chose: it was in here because the C64 had one, in the same way the * prefix is in here because Acorn had it. A machine that never existed is entitled to a voice its author picked on purpose — and the original specification for this one read *4 SIDs/OPL2*, so a synthesiser of the other tradition would not be a betrayal of the plan but the rest of it. Against that: four SIDs were what the machine had sounded like since its first afternoon, they cost almost nothing, and the entire body of C64 music played on them, which is not a small thing to hand back.

C.10 What was decided

On 1 September 2026 the SIDs were turned *off* — not deleted. The OPL2 became the machine's default chip on every host, the menu rows that offered SIDs went away, and one line in `k4510.cfg` gave them back. That is the disposition this project calls OFF, and it exists so that a decision of this size can be tried before it is taken.

On 5 September it was taken: *“nuke anything having to do with SIDs”*. Out went reSID, the four-chip mix with its phase carry, the SID clock select, the settings and their migration, the SID demos and the player, the tune archive, and every register write to \$D400 in the ROM, in EhBASIC and in Mad Pascal's runtime. The machine has one sound chip now and

it is the OPL2, at \$D480, nine FM voices wired the AdLib's way.

Three things fell out of it that are worth recording:

- **The sound sequencer had to learn FM.** It was the SIDs' last real user — BBC BASIC's SOUND, Mad Pascal's Sound and the invaders all reach it — so its four channels became OPL2 voices, with the Beeb's noise channel as a feedback-heavy patch and the other three as plain two-operator tones. The patch is rewritten on every note, because a program may have zeroed the chip for its own purposes and the sequencer must still sound after it.
- **The render seam got a name.** `core/audio.c` is a page of C that owes nothing to any chip: cycles in, samples out at the device's rate, the clock advanced. Everything this appendix is about — the ring, the lead, the filling-in — lives there now, and a future chip plugs into it rather than into the machine.
- **Save states changed format**, and old ones refuse to load rather than pretending.

C.11 And the moral, once more

Read back, the six weeks argue for a conclusion the last two sections then decline to draw. Every fault the sound reported was somewhere else; the chip emulation was never on the critical path; and when the machine was measured properly, the SIDs cost a quarter of one per cent of a frame. The performance case for removing them was nil, and this appendix says so at length.

They were removed anyway, and rightly, because the question was never performance. It was what the machine is. That is the difference between a fantasy computer and a product: a product's parts are decided by measurement, and a fantasy computer's are decided by its author — having first taken the trouble to find out what the measurement actually said, so that the decision is an honest one rather than a rationalised one.

Everything above this section is kept exactly as it was written, because the route matters more than the destination, and a book that qui-

etly rewrote its own history would be worth less than one that admits what it tried.

Disclaimer

The K4510 is a hobby project: a computer that does not exist, built for the pleasure of building it. It is offered to you as a gift, and it comes with exactly the guarantees a gift comes with — none.

In the words of the licence it is distributed under (page 117), and meaning every one of them:

THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM “AS IS” WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

IN NO EVENT WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR REDISTRIBUTE THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM.

In plainer words: nobody promises this machine works, that it will keep working, or that anything it writes to your disk is safe. Keep backups. Do not fly aeroplanes with it, run hospitals on it, or trust it with money. It is a toy, and it is allowed to be wrong.

The same goes for this book: where it and the machine disagree, the machine is right. Nothing here is endorsed by or affiliated with Acorn, Commodore, Microsoft, Digital Research, the MEGA65 project, the Raspberry Pi Foundation, or anyone else gratefully named in these pages. The mistakes are the author’s alone.

Constructive comments can be left at the repository's issues page:

<https://github.com/mlongval/k4510/issues>

All complaints, criticisms and negativity can be addressed to
`/dev/null`

Thank You

The K4510 is a machine that never existed. Almost everything in it that *does* exist was written by somebody else first, and given away. This chapter is the thanks; the Licences chapter (page 117) is the paper-work.

The heart of the machine

Gábor Lénárt (LGB) wrote the 45GS02 CPU core in Xemu, the MEGA65 emulator. It runs here unchanged — not a line altered, not a bug fixed — and every instruction this book describes is his code executing. There would be no K4510 without it.

Jarek Burczyński and **Tatsuyuki Satoh** wrote `fmopl`, the OPL2 emulation in MAME that this machine's YM3812 is — by way of VICE, vendored unaltered. Nine voices of FM that any AdLib register list from 1987 can drive is their work, not ours.

The VICE team, whose decades of emulation scholarship — documentation, test programs, arguments settled in code — this project consulted at nearly every turn.

The tongues

Lee Davison (1966–2013) wrote EhBASIC, the machine's first BASIC and still the one that comes up in ROM. He is not here to be asked about the graphics and sound words bolted onto it; the machine carries his name in its README and its startup banner instead.

Richard T. Russell wrote BBC BASIC, and has kept writing it for forty years. The console edition of his BBC BASIC for SDL 2.0 (BBCTTY) is what runs on the Tube co-processor in Chapter 6, under the zlib licence

he publishes it with. “BBC BASIC” is the name of his interpreter; this book uses it only to say what is running, and claims nothing in it.

Marcelo Dantas (“Mockba the Borg”) wrote RunCPM, which is the whole of Chapter 9: a complete CP/M 2.2 with its own CCP, vendored here unmodified and simply handed a Z80’s worth of address space.

Scot W. Stevenson, **Sam Colwell** and **Patrick Surry** wrote Tali Forth 2 and put it in the public domain — a Forth written to be read, which is why Chapter 7 can be honest about how it works.

Tomasz Biela (“tebe”) wrote Mad Pascal and MADS; the K4510 target in pascal/ is a guest in his compiler. **Wojciech Bociański** (“bocianu”), whose Neo6502 target showed how such a guest should behave.

Steve Wozniak wrote Wozmon in 1976 in 256 bytes, and the monitor in this machine is still recognisably it.

Jim Butterfield (1936–2007) wrote Supermon, and gave it away as he gave away everything: the machine language monitor a generation of Commodore programmers learned on, published in *Compute!* for anyone to type in. Supermon+64 V1.2 is SUPERMON here, ported and not rewritten. **J. B. Langston** restored and commented the source (jblang/supermon64), which is what made porting it a day’s work rather than a month’s; he asks only for the attribution, and it is given gladly.

Michael Steil maintains msbasic, and **Microsoft** released 6502 BASIC 1.1 under the MIT licence in 2025 — between them, the machine’s next native BASIC.

Letters on the screen

Fonts are the part of a computer you look at longest, and clean ones with a clear pedigree are hard to come by.

The Linux kernel contributors — the 8x8 console font (font_8x8.c) that got the text mode on its feet and is still the default.

Paul Gardner-Stephen and **Roman Standzikowski** (FeralChild64) — the clean-room C64/C65 chargen from MEGA65 open-roms, and **Retrofan**, whose PXLfont travels with it by permission.

Ville-Matias Heikkilä (“Viznut”) — unscii, placed in the public domain.

Damian Vila — BESCII, the PETSCII spirit with a clean pedigree, CC0.

Damien Guard drew the ZX Origins fonts — hundreds of 8x8 faces, refined over thirty-five years and given to anyone who wants them. A dozen of them are in the F7 font menu. They are free to use and *not* free to re-host, so this machine does what it does with Commodore’s chargen: it names them and does not ship them. `tools/mkzxfonts.py` turns your own download into screen fonts; without it those menu entries fall back to the default face, and nothing else breaks.

Kenney makes game art and puts it in the public domain, at a scale and a standard that has quietly furnished a decade of small games. The dungeon in the TINY demo — every tile, every little person, and the map they stand on — is his Tiny Dungeon, CC0. The demo exists because the art did. Two more games stand on the same ground: SKYFIRE flies Kenney’s Pixel Shmup ships, and FLUFFY is drawn with **Chloe Wolfe**’s Game Boy platformer set, also CC0. In both cases the art came first and the game was written around it, which is the reverse of the usual order and a much better way to spend an evening.

Ian Schofield wrote Tek40xx, a Tektronix 4010/4014 storage tube on SDL2 that is also a telnet client. It rides along on the machine’s Linux as the second terminal (Chapter 13), built from upstream with one patch of ours.

The bare metal, which the machine no longer stands on

This section is kept because the debt is not cancelled by the code coming out. From August to September 2026 the K4510 ran on a Raspberry Pi 3B+ with no operating system underneath it, and that edition was retired on 2026-09-07 (page 3). Everything below is why it was possible at all.

Randy Rossi wrote BMC64: VICE on a Raspberry Pi with no operating system under it, 50 Hz smooth scrolling and input latency measured in single frames. His initials were in that edition’s name — the **BMC-K4510**

was the K4510 on the bare-metal platform he proved out — and it is the direct reason the port existed at all: the route to the Pi was always meant to be BMC64’s `emux_api` seam, and the bare-metal case it proved is what made it thinkable. He also built the **VIC-II Kawari**, a modern drop-in VIC-II with video modes the original never had: the demonstration that you may extend an 8-bit machine’s video chip and still be working inside the tradition rather than outside it. VICKY is a more reckless cousin of that idea. **minch** (aminch) has taken BMC64 over from him and carries it onto the newer Pis; the project is in good hands.

Rene Stange wrote Circle, the bare-metal C++ environment that was the entire reason a Raspberry Pi 3B+ could boot straight into this machine with no operating system under it. **Xalior** wrote `circle-libsdl2`, which let the desktop emulator move to the Pi essentially as written.

The workshop

The machine is built with **cc65** (compiler, assembler, linker and runtime for the whole system ROM), Marco Baye’s **ACME** (Wozmon and the demos), Zsolt Soós’s **64tass** (Tali Forth and Supermon), **NASM**, **GCC** and **GNU Make**; it shows itself to you through **SDL2** — Sam Lantinga’s library and its contributors’, which is the window, the keyboard and the sound on the desktop, and through Xalior’s port, on the Pi as well. This book is set in **XeLaTeX** with **Clear Sans** and **Iosevka**, and every screenshot in it was captured from the machine actually running — a picture that cannot be produced fails the build.

Heritage

Some debts are not code.

Acorn Computers — the Tube, the sideways-ROM model, the `*` command prefix, and the idea that a second processor should be a normal thing to own. The Beeb’s ghost is all over this machine.

Commodore — the other half of its soul: PETSCII, the SID, and the line from the PET to the C65 that stopped one machine short of this one.

The MEGA65 project — for keeping the 45GS02 and the C65 dream alive in real silicon, and for open-roms.

Kim Lemon and the Lemoners — Lemon64 has been the C64's front door since 1998: the games database, the reviews and scans, the music, and a forum that actually answers. Twenty-eight years of one person's dedication to a machine's community, and a good half of the small facts this project needed came from there.

The Meatloaf project (idolpx) — the C64 cartridge that made a URL a file name, which is the whole of this machine's rule for the network: name it, and it is fetched. **The FujiNet project** — the network device the Atari got first, whose N: device this machine imitates and whose TNFS servers it speaks to; `CD tnfs://fujinet.online/` is a directory on theirs.

Paul Scott Robson wrote the Neo6502's firmware — the operating system and BASIC that make an Olimex Neo6502 a computer rather than a board — and the handbook that documents it. That book is the one this one is modelled on, down to the page size, and the rule that every example must be run and photographed before it may be printed is his. The other model is the **MEGA65 User's Guide**.

The High Voltage SID Collection and the composers in it, who kept thirty years of a machine's music alive and free to listen to. The K4510 played it for one summer, through a SIDPLAY that retired with the chips. None of that music was ever distributed with this machine; it is theirs.

The project's orchestrator is Michael Longval — Doc — who decided what the machine is, what it borrows and what it refuses, what gets built next and what gets thrown away, and read every page of this book against the machine.

And the machine's other author. Most of the K4510's own code — VICKY, SHEILA, the DMA engine, the ROM, K/OS, the Tube ULA, the editors, RX — was written in conversation with Claude (Anthropic's Claude Code), over several months of long sessions with a build log to prove it. The design decisions are the author's; a great deal of the typing was not.

And whoever is missing

This list was assembled by hand and is certainly incomplete. If your work is in this machine and your name is not on this page, that is an error and not a judgement — open an issue at <https://github.com/mlongval/k4510> and it will be fixed in the next printing.

Licences

The Thank You chapter (page 111) is the thanks; this is the record. The authoritative copy is `LICENSES.md` in the repository — if the two ever disagree, that file wins.

The project itself

The K4510 as a whole is distributed under the **GNU General Public License, version 2 or (at your option) any later version**. Everything written for this project — the machine, VICKY, SHEILA, the DMA engine, the MATH unit, the system ROM and K/OS, JIM, the Tube ULA, the network layer, the editors, the demos, RX and the Pascal target — is Copyright © 2026 Michael Longval and is released under those terms. The choice is not really a choice: the CPU core and the OPL2 emulation are GPL-2.0-or-later, and a work containing them must be too.

Code inside the machine

- **Xemu 65xx/45GS02 CPU core** — Gábor Lénárt. `core/xemu/`: `cpu65.c`, `cpu65.h`, `cpu65_mega65_timings.h`, `cpu65ce02_disasm_tables.c`, used unchanged. *GPL-2.0-or-later*.
- **fmopl (YM3812/OPL2)** — Jarek Burczyński and Tatsuyuki Satoh, from MAME by way of VICE. `core/opl2/`, vendored unaltered. *GPL-2.0-or-later*.
- **reSID** — Dag Lem, as shipped in VICE 3.3, was `core/resid/` until 2026-09-05 and is no longer in the tree (Appendix C). *GPL-2.0-or-later*; recorded here because a vendored component that quietly disappears is worse than one recorded as gone.
- **BBC BASIC for SDL 2.0, console edition (BBCTTY)** — Richard T. Russell. `tube/`, vendored and altered (see `tube/ALTERED.md`). *zlib licence*.

“BBC BASIC” is the name of Mr Russell’s interpreter, which he publishes under his own arrangement with the BBC. This project has not sought, and does not hold, any licence to the name: it is used here only to identify the interpreter, which is why this book says “Richard Russell’s BBC BASIC” and not “the K4510’s”.

- **EhBASIC 2.22** — Lee Davison, in the ca65 form from jefftranter/6502. `basic/basic.asm`. *Free for non-commercial use only*; derivatives must carry the words “Derived from EhBASIC” — see `basic/README-EhBASIC.txt`. It ships as a separate program (`fs/LANG/EHBASIC/ehbasic.prg`) and is not linked with the GPL code.
- **RunCPM** — Marcelo Dantas. `cpm/src/`, vendored unmodified (`cpm/VENDORED-FROM.txt`). *MIT*.
- **Tali Forth 2** — Scot W. Stevenson, Sam Colwell, Patrick Surry. `forth/tali/`, vendored unmodified. *Public domain*.
- **Wozmon** — Steve Wozniak’s 1976 monitor, reimplemented here for the 45GS10. Apple published the original listing; this machine’s version is the project’s own code under the project licence.
- **cc65 runtime** — the ROM and every `.prg` are linked against `none.lib`. *cc65’s zlib-style licence*.
- **Microsoft 6502 BASIC** — *MIT*, released by Microsoft in 2025, via `mist64/msbasic` at a pure-Microsoft configuration. `basic/msbasic/`, vendored unmodified — and only the files a pure build assembles: none of the per-manufacturer material is here, which is the line that keeps the build resting on the MIT release alone.
- **Supermon+64 1.2** — Jim Butterfield, restored and commented by J. B. Langston (`jblang/supermon64`), ported to the 45GS10. Butterfield published it for anyone to use; Langston asks for attribution.

Fonts

- **Linux kernel 8x8 console font** — `data/font8.bin`, built by `data/mkfont.py` from `lib/fonts/font_8x8.c`. *GPL-2.0*. This is the default text font.

- **MEGA65 open-roms chargen + PXLfont 2.3** — data/fonts/openroms/. Clean-room C64/C65 character shapes; PXLfont is Retrofan's, included with open-roms' recorded permission. *LGPL-3.0-or-later* — 8x8font.png is shipped as the editable source, for LGPL compliance.
- **unscii-8** — Ville-Matias Heikkilä. data/fonts/unscii/; font8-unscii.bin is a generated drop-in alternative to data/font8.bin. *Public domain*.
- **BESSCII v3** (Mono + source) — Damian Vila. data/fonts/bescii/. *CCO-1.0*.
- **Clear Sans** (Intel, *Apache-2.0*) and **Iosevka** (Belleve Invis, *SIL OFL 1.1*) — the two faces this book is set in. They are not part of the machine.

No Commodore ROMs here. Until 2026-08-23 the text font was derived from a Commodore 64 character ROM. That file and every derivative were removed from the repository *and from its history* before it was made public. If you own a C64 and want the real thing, drop your own chargen.bin into /SYSTEM/ETC: the machine will use it, and .gitignore will keep it out of the repository. That copy is yours, not ours to publish.

The Linux the machine boots on

The live image is a Debian, assembled by linux/build-live.sh from Debian's own archive, so every package in it carries its own terms and Debian's copyright files travel with it. One thing is built from source into it:

- **Tek40xx** — Ian Schofield's Tektronix 4010/4014 terminal, built from upstream with one patch of ours. *GPL-3.0*. It is a separate program on the Linux side, not part of the emulator.

The Tektronix plot files in linux/tek40xx/plt/ are Tek40xx's own gnuplot examples plus a page generated here. The vintage cassette plots that circulate with *other* Tektronix emulators are marked personal-use-only

by their authors and are deliberately *not* vendored; the README says how to fetch them for yourself.

The Raspberry Pi build was here until 2026-09-07 and needed two components that never lived in this repository: **Circle** (Rene Stange, *GPL-3.0*) and **circle-libsdl2** (Xalior, *zlib*, with a *GPL-3.0* linker script). A kernel8.img built from those was *GPL-3.0* as a whole, which *GPL-2.0-or-later* code permits. The tag alpha-0.5 is the last release that shipped one.

Build tools

Not distributed with the machine, but required to build it: GCC and GNU Make (*GPL*), cc65 (*zlib-style*), 64tass (*GPL-2.0*), NASM (*BSD-2-Clause*), SDL2 (*zlib*), Python 3 (*PSF*), and for this book XeLaTeX (*LPPL*). Mad Pascal and MADS (Tomasz Biela, *MIT*) are needed only if you compile Pascal; the K4510 target in pascal/ is installed into your own Mad Pascal checkout.

What is deliberately not shipped

- **The CP/M system disk.** RunCPM's DISK/A0.zip — Digital Research's ASM, MAC, DDT, STAT, SUBMIT and third-party tools — installs into fs/CPM/A/0/ for your use but is not committed: its files have mixed provenance and this repository is public. The same goes for WordStar, Turbo Pascal 3 and MBASIC, which the K-*.SUB launchers know how to start but which you must supply yourself.
- **ZX Origins fonts.** Damien Guard's faces are free to use and *not* free to re-host, so the machine names them in the F7 font menu and does not ship them: tools/mkzxfonts.py turns your own download into screen fonts, and without it those entries fall back to the default face.
- **A Commodore character ROM** — see the note above.

- **Your `STARTUP.BAT`** — yours, not the repository's (copy `/SYSTEM/ETC/STARTUP.SAMPLE` to start one).

If you redistribute the machine

Ship the sources you built from, keep `LICENSES.md` and this chapter with them, keep `8x8font.png` beside the open-roms font, and remember two things that are easy to forget: EhBASIC is non-commercial-only, and “BBC BASIC” is the name of Richard Russell’s interpreter rather than of anything made here — describe it as his, as this book does, and give whatever you build on it a name of its own.